Laborator 01 - Eagle 101

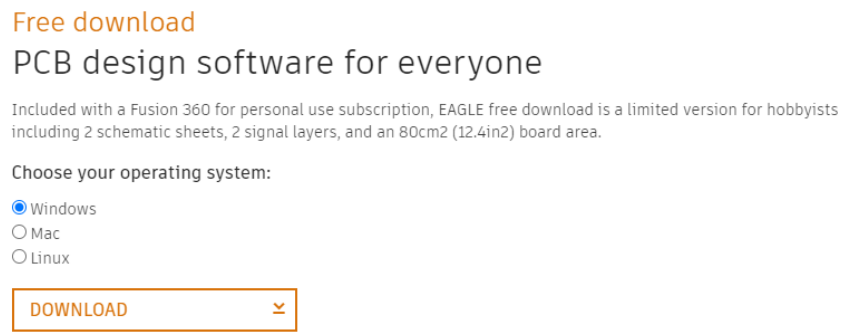
Obiective

Prima secțiune de laboratoare se vor concentra pe proiectarea de circuite, folosind EAGLE (**E**asily **A**pplicable **G**raphical **L**ayout **E**ditor), pentru a pregăti realizarea lor fizică.

EAGLE este o aplicație software realizată de către Autodesk [https://www.autodesk.com/products/eagle/overview/]. Cu ajutorul său, vom putea proiecta circuite electronice, precum și realiza traseele folosite la cablajul imprimat. EAGLE necesită o licență pentru utilizare, dar poate fi folosit gratuit cu anumite restricții, cum ar fi dimensiunea maximă a board-ului și numărul de layere. Unul dintre marile avantaje pe care le oferă este faptul că există numeroase biblioteci cu piese predefinite. O parte dintre aceste biblioteci sunt realizate chiar de către comunitate, printre cele mai notabile fiind bibliotecile puse la dispoziție de: SparkFun [https://github.com/sparkfun/SparkFun-Eagle-Libraries], Adafruit [http://www.ladyada.net/library/pcb/eaglelibrary.html], Element14 [http://www.element14.com/community/community/cadsoft_eagle/eagle_cad_libraries] etc.

Instalare și folosire

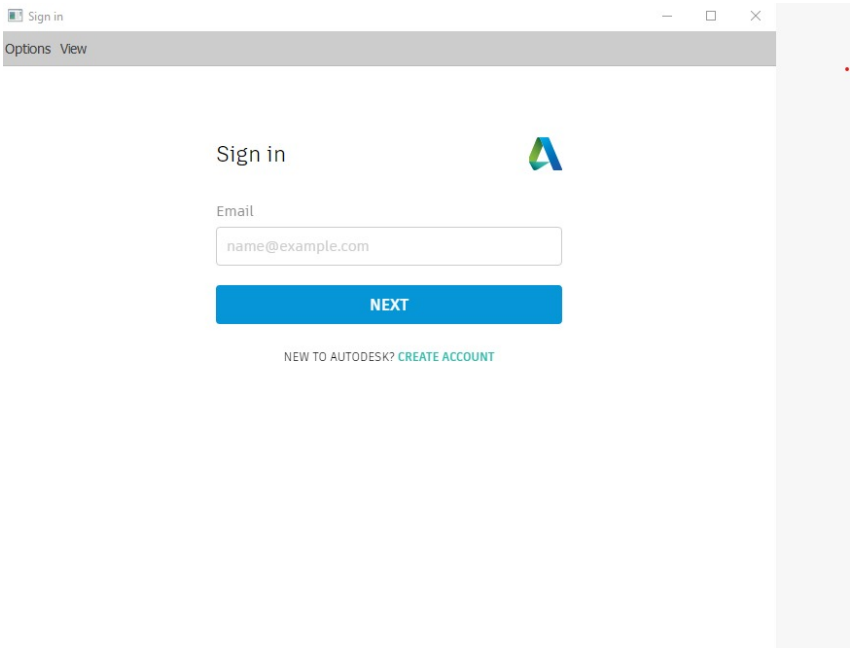
Pentru a instala EAGLE, trebuie să descărcați installer-ul potrivit pentru sistemul vostru de operare de pe pagina de downloads [https://www.autodesk.com/products/eagle/free-download/].



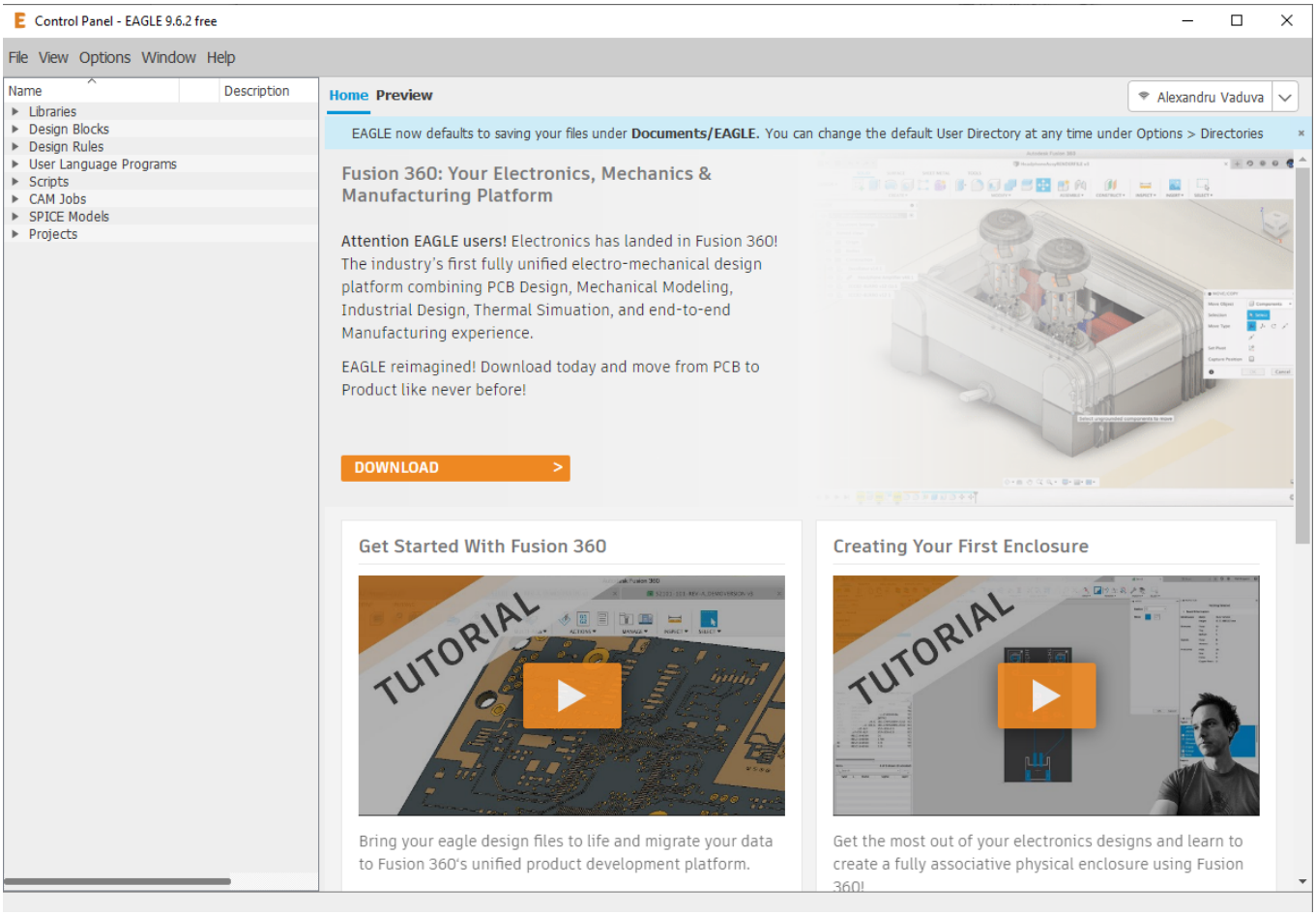
Pe Ubuntu este disponibil şi prin intermediul apt:

```
sudo apt-get install eagle
```

Pentru a utiliza EAGLE aveți nevoie de un cont pe situ-ul Autodesk [https://www.autodesk.com/]. La prima rulare EAGLE veți fi nevoiți să vă autentificați.



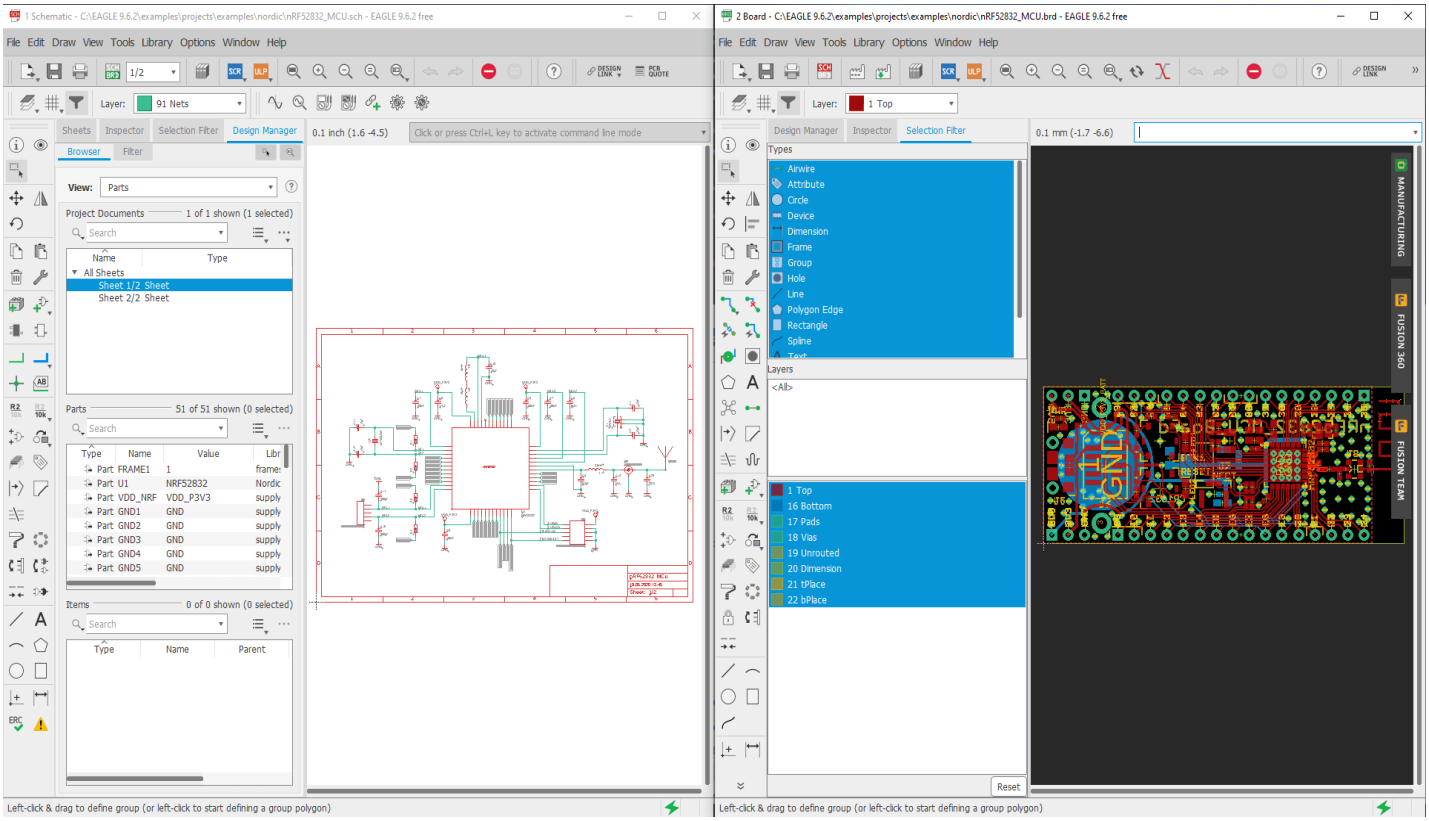
După instalare, deschiderea aplicației vă va da acces la un Control Panel (vezi figura de mai jos), de unde puteți crea proiecte și încărca biblioteci.



Ce se poate face cu EAGLE?

- Puteți crea schematicul aplicației (similar cu ceea ce făceați in Falstad). Fișierele salvate vor avea extensia .sch.
- Din schematic puteți genera Board-ul PCB-ului. Board-ul este legat de Schematic; la nevoie poate fi re-generat. Fișierele salvate vor avea extensia .brd.
- EAGLE poate face verificarea schematicului și a boardului pentru diferite tipuri de erori (fire lăsate în aer etc.) care sunt foarte utile înainte de realizarea plăcuței fizice.
- Puteți crea piese custom pe care să le folosiți în noile proiecte (Schematic și Board).

După ce creați board-ul pentru o schemă electronică, puteți să printați cablajul imprimat pentru a-l folosi la realizarea plăcuțelor voastre. Astfel, se poate observa că un proiect EAGLE complet se realizează din 2 pași: schematic și board, ca în figura de mai jos:



De ce am nevoie să încarc biblioteci? Pentru că atunci când faceți proiectarea cunoașteți deja piesele pe care le folosiți (cele cumpărate din comerț), care au anumite dimensiuni („footprint”) de care trebuie să țineți cont la generarea board-ului.

De ce am nevoie să fac piese customizate? Pentru că este posibil ca pentru unele piese pe care doriți să le folosiți / le-ați cumpărat deja să nu găsiți bibliotecile corespunzătoare. Astfel, EAGLE permite realizare propriilor biblioteci și încurajează upload-ul acestora pe site-ul lor.

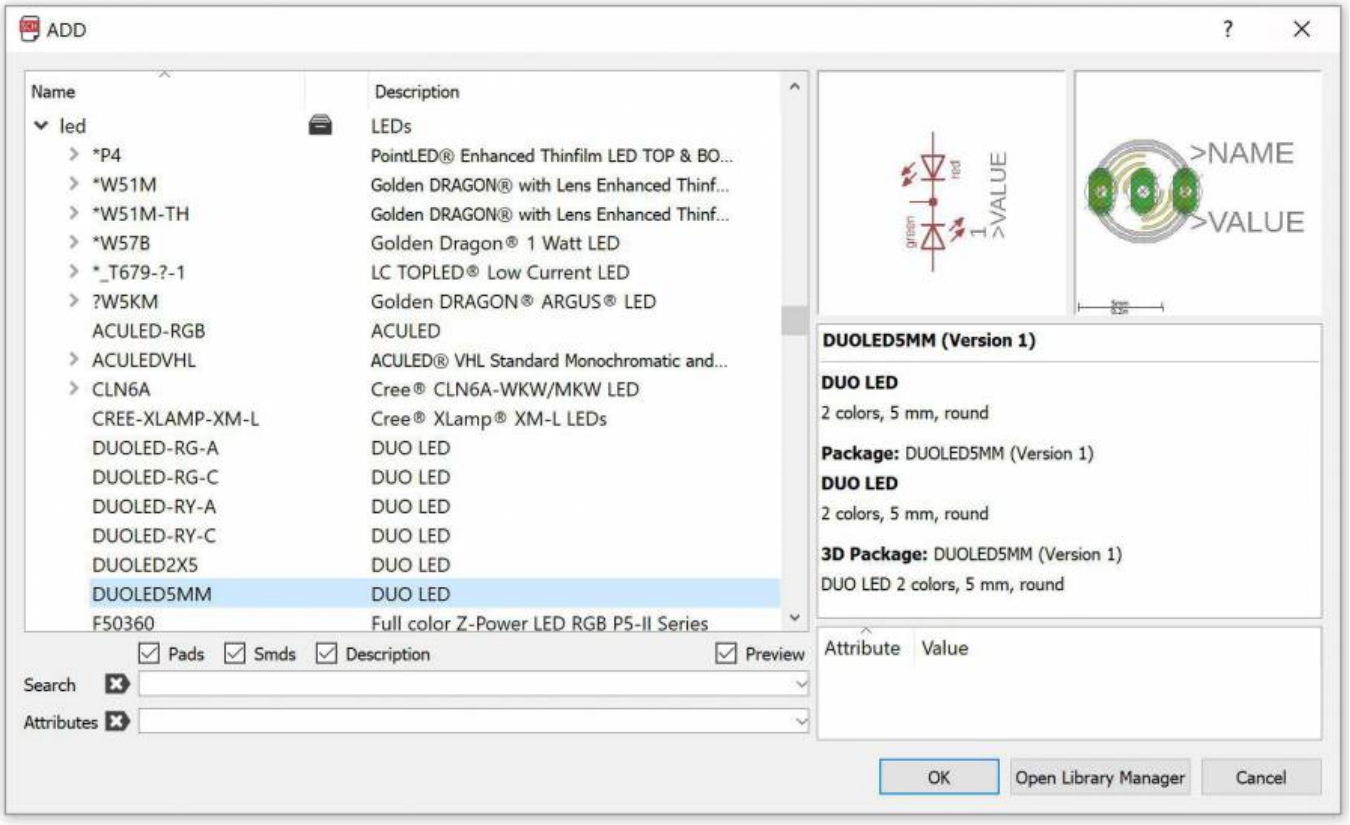
Folosirea editorului schematic

Pentru a crea o noua schema :

- **File → New → Schematic**

Adăugarea pieselor în schemă

Pentru a adăuga piese în schemă este folosit butonul **Add** (). Va apărea o fereastră ce afișează listele de biblioteci din sistem, de unde puteți să selectați piesa dorită. Vor fi afișate informații despre piesă, cum va arăta piesa în schema și cum va arăta pe PCB. De asemenea, se pot căuta piese:





Pentru a adăuga piesa apoi în schemă, se face click în locul unde vrem să punem piesa. Înainte de a poziționa piesa, puteți folosi click-dreapta pentru a o roti. De asemenea, după adăugare piesa rămâne selectată astfel încât se pot adăuga cu ușurință mai multe piese de acelasi tip.

Inițial ele vor avea nume generice, însa acestea pot fi redenumite. Acest lucru se face prin Click-dreapta pe piesă → **Name**. Pentru a fi reprezentată și o valoare legată de piesă, se face Click-dreapta pe piesă → **Value** și este introdusă valoarea dorită.

Selectarea unei componente se face cu click stânga!

Daca schema devine prea aglomerată, pot fi folosite uneltele zoom:  . Acestea pot fi folosite pentru a face zoom in, zoom out, pentru a încadra toata schema în ecran sau pentru a încadra o anumita zona în ecran.

Așezare piese în schematic

În partea de sus a toolbar-ului din stânga sunt unelte ce pot fi folosite pentru a muta și așeza piese în scenă.



- **Group**
- **Move**
- **Mirror**
- **Rotate**
- **Copy**
- **Paste**
- **Delete**
- **Change**

Legături între piese

După ce ați adăugat piesele în scheme, trebuie făcute legăturile între ele. Pentru a vedea mai ușor pinii pieselor, poate fi activat layer-ul **Pins**:

- Click pe butonul **Layer settings**()
- Selectarea/deselectarea layerului prin click pe numarul lui(93 în cazul ăsta).
- Click **Apply**

Pentru a crea o legatură:

- Click pe butonul **Net**( ). **Doar NET are semnificație electrică, nu și WIRE.**
- Se pozitionează crosshair-ul pe pinul primei piesei și se face click
- Se poziționează crosshair-ul pe pinul piesei destinație și se face click
- Pot fi adăugate oricâte puncte intermediare
- Poate fi legat la o conexiune deja existentă.

Etichete

Dacă vom avea multe legături între piese, schema ar putea deveni foarte aglomerată. Pentru a elimina confuziile putem eticheta legăturile.

Se poate face acest lucru folosind butonul Label ( ). Pentru a adăuga o eticheta se apasă click-stânga pe conexiunea dorita și se poziționează după preferințe. Eticheta va avea numele implicit al conexiunii. Pentru a o redenumi se apasă click-dreapta pe legatură → **Name**.

Toate legăturile conectate între ele vor avea aceeași etichetă. De asemenea, putem atașa mai multe etichete aceleiași legături sau unor legături care sunt conectate între ele. Toate aceste etichete vor avea același nume în mod automat.

Magistrala

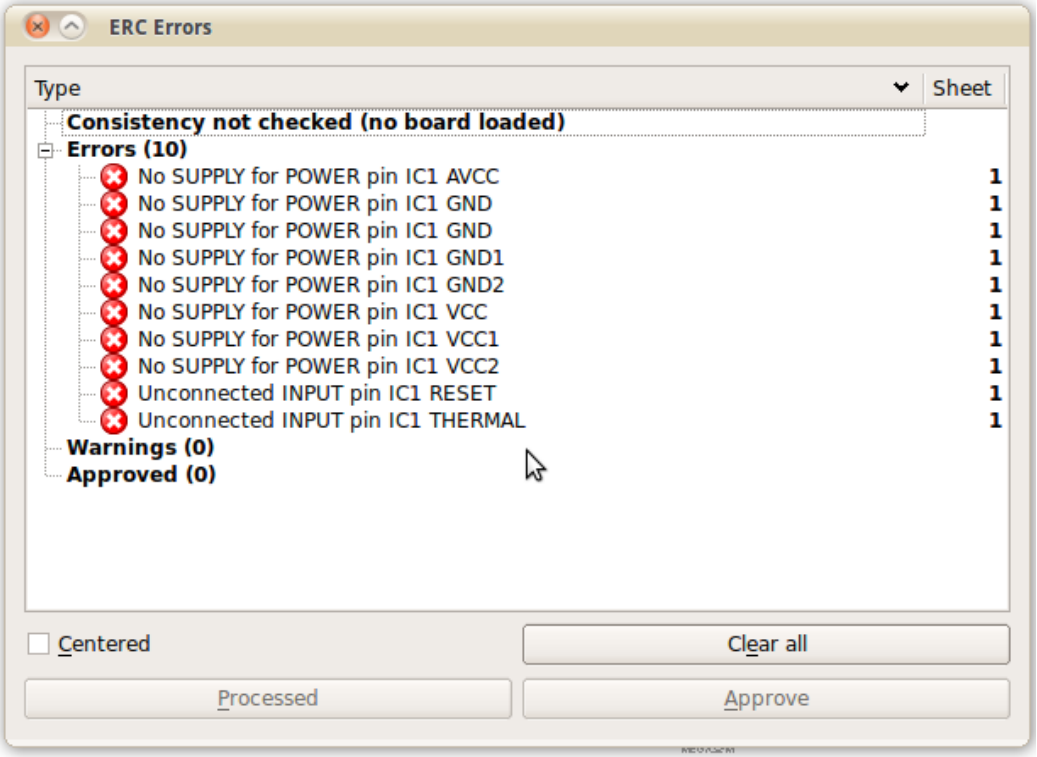
Magistrala este un tip special de conexiune, ce reprezintă un set de fire legate logic între ele. Un exemplu ar fi o magistrală de date pe 16 biți: *DATA[0:15]*. Pentru a reprezenta o astfel de legatură în Eagle, se foloseste butonul **Bus** ( ). Se pot lega apoi conexiune normale la el.

Pentru a specifica conexiunile care pot face parte din magistrală, aceasta trebuie denumită în următoarele moduri:

- **NUME[0..x]** - pentru conexiuni denumite NUME0, NUME1 etc.
- **AAA,BBB** - pentru conexiuni numite AAA sau BBB.

Electrical Rule Check

După ce am terminat schema, comanda *ERC* ne poate testa circuitul pentru erori. Aceste lucru se poate face folosind butonul **ERC**( ):



Board Layout

Fișierul board poate fi generat automat din fișierul schematic sau poate fi construit de la zero. Avantajul generării din schematic constă în faptul că piesele vor fi deja adăugate, împreună cu semnalele dintre ele. Componentele unui fișier .brd sunt:

- Componente. Sunt piesele ce compun circuitul.
- Semnale. Reprezintă legăturile logice din circuit. Ex. Pinul PD7 de la atmega este legat la pinul A de la o poartă logică.
- Rute. Reprezintă traseul fizic de placa ce face legatura între pinii legați printr-un semnal.

Pentru a trece de la schematic la board folosiți:

File → Switch to board

Pentru a lucra cu mai multe componente deodată folosiți unealta **Group** ( ) după care selectați modificarea pe care o vreți efectuată (Ex: Move) și din meniul de context accesat dând click dreapta pe grup alegeți acțiunea per grup (Ex: Move: Group).

Schematicul și board-ul generat fac parte din același proiect. De multe ori este nevoie să ne dăm seama ce reprezintă un fir din board sau care e traseul din board corespunzător unei conexiuni din schematic. Pentru a face acest lucru, putem să folosim opțiunea „Show” din tool box, care permite ca la selectarea unei componente în schematic să facă highlight la componenta corespondentă din board și invers. Opțiunea „Show” se selectează separat pentru schematic, respectiv pentru board. În plus, având opțiunea activă, selectarea unui pin în schematic va face highlight la toți pinii care sunt conectați pe acea rută (similar pentru board).

Ca o alternativă la opțiunea „Show”, pentru a vedea dacă un pin este conectat la o componentă, puteți încerca mutarea piesei. Spre deosebire de cum era în cazul simulatorului Falstad, EAGLE permite mutarea conexiunilor odată cu piesele.

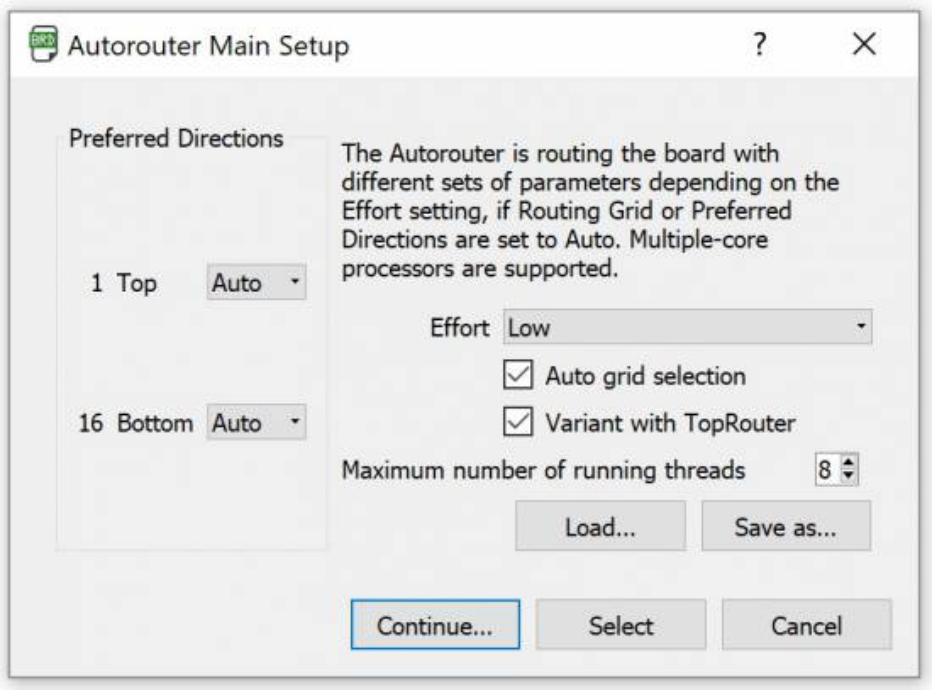
Rutare

Puteți observa faptul că la generarea board-ului piesele sunt conectate prin niște fire galbene, subțiri, numite airwires. Acestea nu reprezintă decât faptul că există o conexiune între acele piese în schematic, dar nu reprezintă traseele de care avem nevoie. Ne vor ajuta, însă, la crearea rutelor între componente.

Autoroute

Rutarea este facuta automat de catre eagle pe baza unui algoritm. Acest algoritm poate fi parametrizat prin asignarea unor costuri elementelor. Rutarea automata poate esua uneori sau poate produce rezultate proaste.

Pentru a face autorutare din eagle dați click pe butonul **Autorouter** ( ). Din meniul lansat puteți alege diversele proprietăți pe care să le aibă în vedere rutarea, una din cele mai importante este cum să trateze trasarea pe suprafața de sus și cea de jos a plăcii.



Rutare manuala

De multe ori autorutarea poate să genereze rezultate în parte ineficiente. Astfel, este indicat să folosiți o combinație de autorutare cu rutare manuală pentru a obține cele mai bune efecte.

Se ruteaza fiecare semnal manual folosind unealta **Route** ( ) . Pentru a elimina o rută folosiți unealta **Ripup** ( ).

Ratsnest

Pentru a verifica faptul că toate Net-urile sunt rutate din schematic, folosiți opțiunea RATSNEST . Dacă totul a mers bine și ați rutat toate airwire-urile, veți vedea mesajul următor în status box-ul din stânga-jos: „Ratsnest: Nothing to do!”.

Design Rule Check

Similar cu **ERC**-ul (Electrical Rule Check), există DRC (Design Rule Check ) care este o unealtă ce face verificări asupra circuitului conform anumitor parametri. De aici se pot seta verificări pentru a vedea dacă două linii sunt prea aproape, se poate mări dimensiunea găurilor de burghiu, cât și cea a pad-urilor. În principiu de aici se pot face diverse modificări pentru a ne asigură că circuitul reprezentat poate fi implementat folosind uneltele avute la dispoziție. Pe lângă asta, puteți încărca și un set de reguli predefinite.

Cele mai comune probleme care pot apărea sunt: suprapunerea a două sau mai multe rute (numită **overlap**), trecerea unei rute prea aproape de un pad / via, trecerea unei rute prea aproape de marginea board-ului etc.

Pentru mai multe detalii puteți consulta următorul link. Conține descrierea tipurilor de piese utilizate pentru realizarea design-ului plăcuțelor electronice: List of integrated circuit packaging types [https://en.wikipedia.org/wiki/List_of_integrated_circuit_packaging_types?fbclid=IwAR0xorPt1fOFQKh0rlgFZLCiujT1WUUhSijikPyHEHCdgmitqiYuqxpCI8o]

Exerciții

1. Instalați biblioteca Adafruit din repository-ul GitHub de aici [https://github.com/adafruit/Adafruit-Eagle-Library], într-un folder care conține numele vostru. Folosiți una dintre piesele din această bibliotecă (adăugați-o într-un schematic nou).
2. Deschideți Falstad [http://www.falstad.com/circuit], la circuitul Circuits → Logic Families → DTL → DTL NAND.
    a. Realizați schematicul pentru acest circuit.
    b. Generați board-ul PCB folosind pașii descriși la secțiunea aferentă. Aranjați componentele în zona delimitată board-ului. Realizați rutarea între componente folosind doar layer-ul „bottom”.
    c. Realizați verificarea ERC.
    d. Realizați verificarea DRC folosind opțiunile:
    - verificarea pentru distanțe dintre trasee să fie minim *10 mil* între oricare două componente.
    - verificarea pentru găuri de burghiu să fie tot de *10 mil*.
    - verificarea pentru dimensiunea pad-urilor să fie cu 50% mai mare decât este setată implicit.
3. Realizați un schematic în care folosiți un microcontroller *ATMEGA328P* care va primi pe pinii PC0-3 un număr de la un switch(*DIP-04SMD*, ambele se găsesc în biblioteci de la SparkFun: *Sparkfun-IC-Microcontroller* respectiv *Sparkfun-Switches*) și va transmite folosind o magistrală de pe pinii PD0-7 către un afișor 7-segmente (*7-SEGMENT-DISPLAYPTH)*, tot din biblioteca SparkFun). Generați board-ul și realizați rutarea.
    - Hint: Biblioteca de la SparkFun se poate adăuga pentru utilizare prin meniul **Library** → **Use** (daca nu gasiti, descarcati bibliotecile de la https://github.com/sparkfun/SparkFun-Eagle-Libraries [https://github.com/sparkfun/SparkFun-Eagle-Libraries]).
    - Hint: La magistrală trebuie etichetați identic liniile de la pinii PD0-7 și A-DP.

Resurse

- Documentația oficială pentru EAGLE [http://eagle.autodesk.com/eagle/documentation]
- Tutorial SparkFun pentru adăugarea de biblioteci noi [https://learn.sparkfun.com/tutorials/how-to-install-and-setup-eagle/using-the-sparkfun-libraries]
- Tutorial SparkFun pentru schematic [https://learn.sparkfun.com/tutorials/using-eagle-schematic]
- Tutorial SparkFun pentru board [https://learn.sparkfun.com/tutorials/using-eagle-board-layout]

icalc/laboratoare/laborator-01.txt · Last modified: 2022/03/01 12:24 by giorgiana.vlasceanu

Laborator 02 - Eagle 102 - Piese Custom

Eagle vine cu multe biblioteci, fiecare cu foarte multe piese, dar e posibil să nu găsiți piesa de care aveți nevoie în bibliotecile standard. În acest caz aveți două opțiuni:

- Căutați o bibliotecă ce conține piesa de care aveți nevoie (ex. biblioteca SparkFun [https://github.com/sparkfun/SparkFun-Eagle-Libraries], ce conține piese ce pot fi găsite pe site-ul www.sparkfun.com [http://www.sparkfun.com])
- Creați piesa în Eagle

Atenție la bibliotecile găsite; verificați-le înainte să le folosiți. Ele trebuie să coincidă cu datasheet-ul piesei pe care vreți să o folosiți.

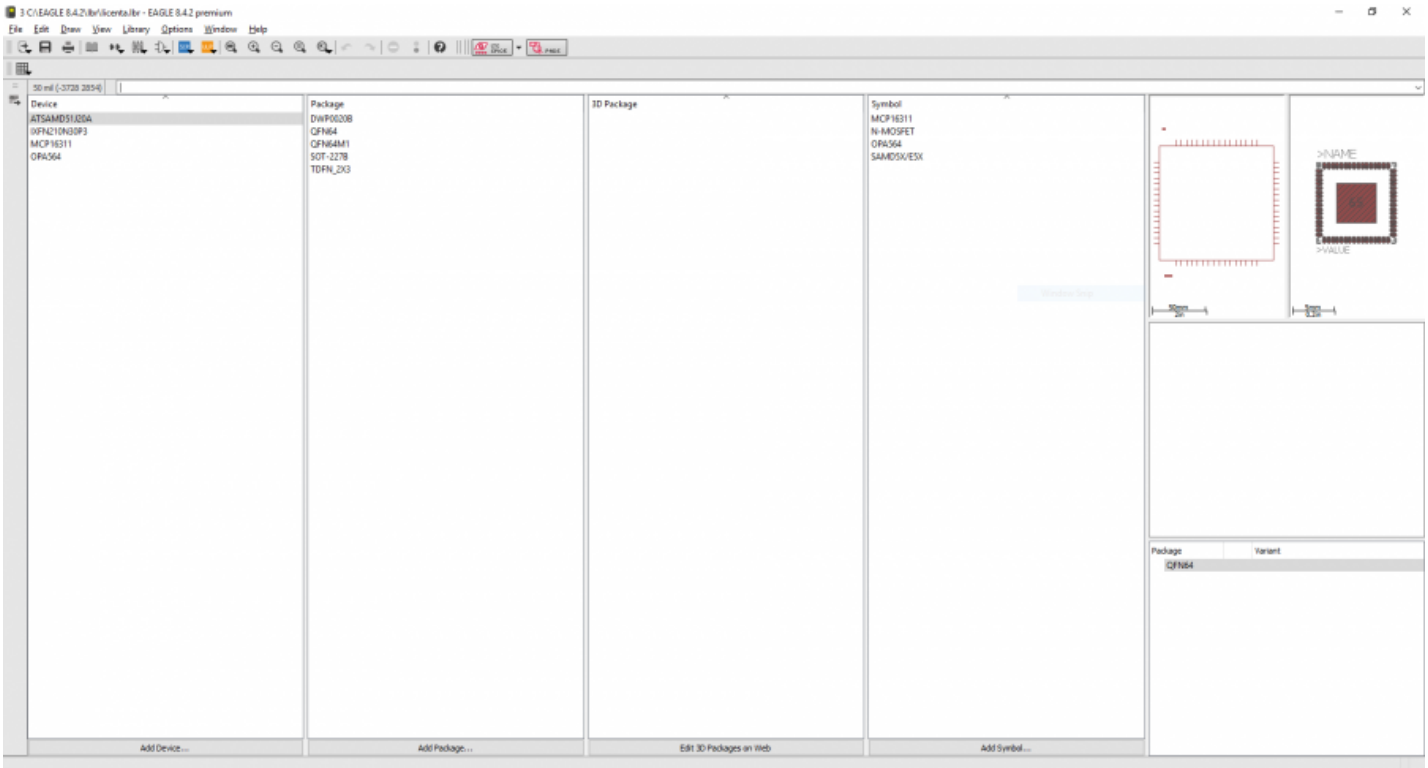
Puteți pune o piesa nouă în bibliotecile deja existente în EAGLE, dar este recomandat să le organizați în biblioteci noi. Pentru a crea o nouă bibliotecă:

- **File→New→Library**

O bibliotecă este compusă din *Device-uri*, iar Device-ul este compus din urmatoarele părți:

- *Symbol*: Simbolul ce va fi folosit în schemă - definește forma în schemă și pinii device-ului;
- *Package*: Dimensiunile fizice ale device-ului - trebuie să coincidă cu cele din datasheet;
- *3D Package*: Reprezentarea 3D a device-ului - utilă pentru observarea dimensiunilor plăcuței proiectate (nu este obligatoriu).

Pentru a adăuga o piesă nouă într-o bibliotecă avem mai multe variante. Indiferent de varianta aleasă, trebuie ca mai întâi să deschidem biblioteca și vom observa *Table of contents*:



Adăugarea unei componente trebuie să se facă prin crearea unui *Symbol*, a unui *Package* și apoi a unui Device ce le va lega pe celelalte două.

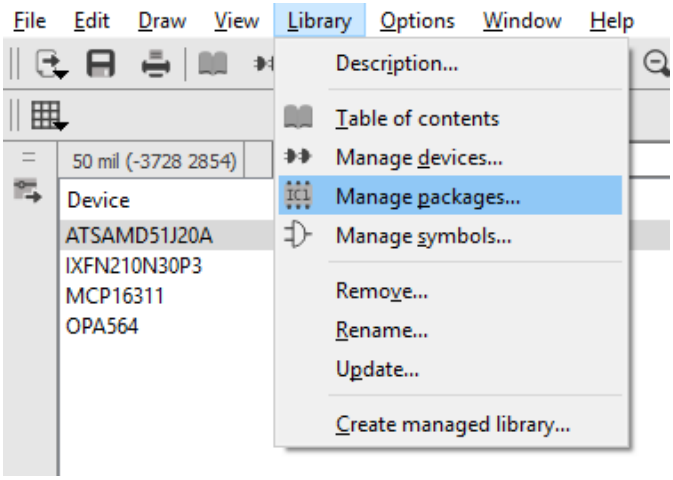
Posibilități de adăugare symbol/package/device:

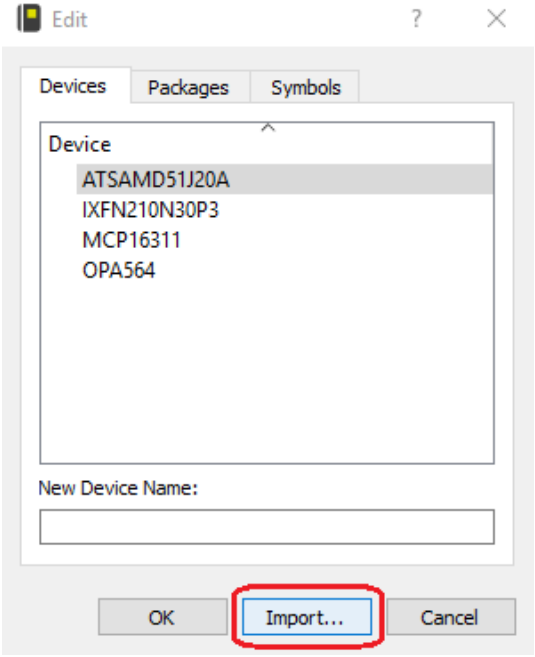
1. Ordered List ItemDin *Table of Contents*: se pot observa în imaginea de mai sus, în partea de jos, *Add Device…*, *Add Package…* și *Add Symbol…*;

2. Din bara de opțiuni din partea de sus:

3. Din meniul *Library*:

O altă facilitate importantă a fost introdusă recent în ultimele versiuni de Eagle: importarea device-urilor, a package-urilor sau symbol-urilor din alte biblioteci deja existente:





Symbol

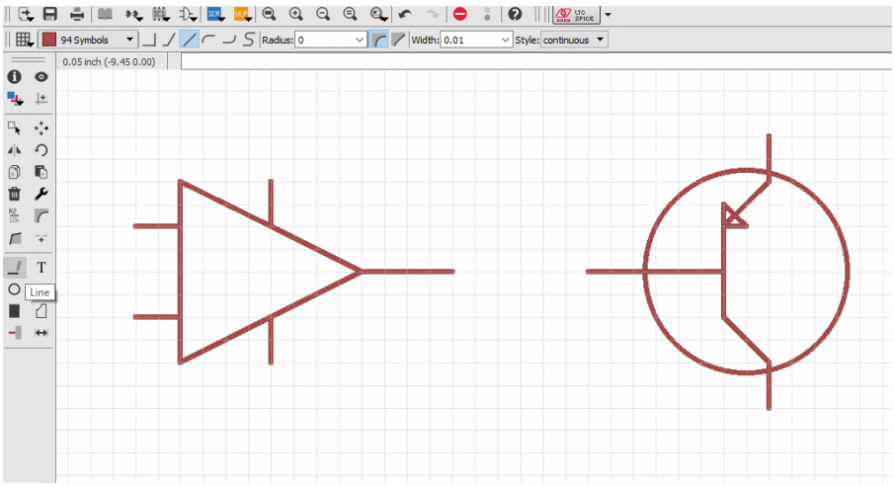
Aici este definit modul în care va arăta piesa în schematic. De obicei, este prezentat în datasheet un simbol recomandat, dar nu este obligatoriu ca simbolul definit în eagle să arate ca cel din datasheet. Pentru desenarea simbolului se vor folosi primitive grafice precum linii și arce de cerc și este obligatoriu să utilizăm layer-ul *94 Symbols* - selectat by default.

Unelte disponibile:



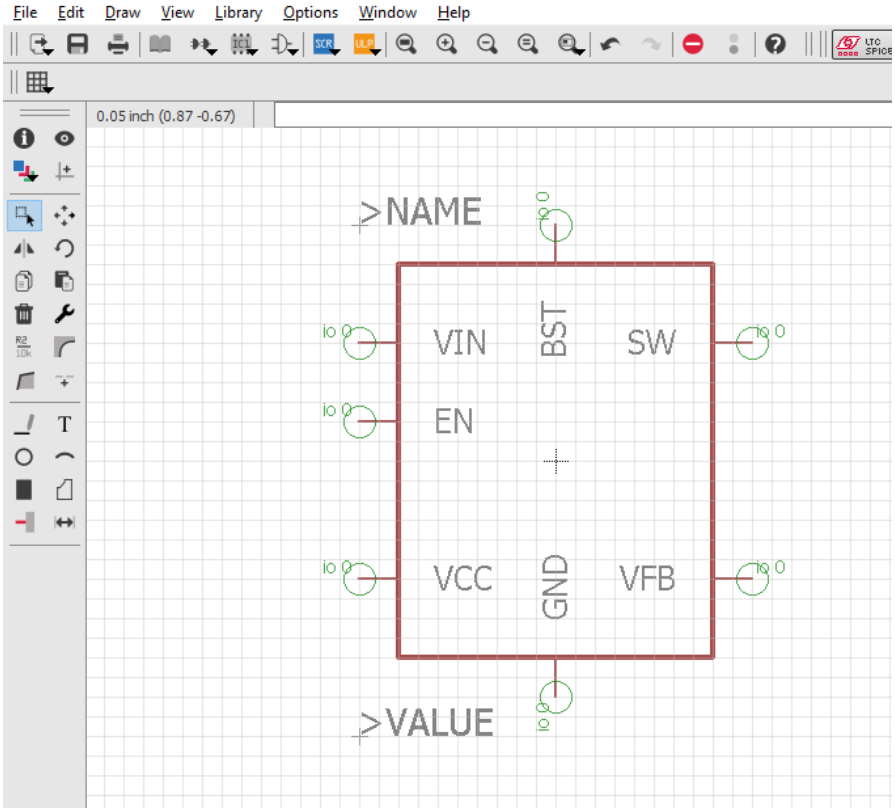
- **1 (Pin)**. Adaugă un pin pe device. Este util să fie definite asemănător cu datasheet-ul, dar nu este necesar. Legatură dintre ele și punctul de contact va fi făcut la *Device*. Inițial îi este dat un nume nedescriptiv; pentru a redenumi pinul, click dreapta pe el și **Name**.
- **2 (Line)**: Desenează una sau mai multe linii; pentru a termina o linie se face Dublu-click sau se apasă **Esc**;
- **3 (Text)**: Adaugă text în simbol;
- **4 (Arc)**: Deseneaza un arc. Mai întâi este definit cercul din care face parte arcul și punctul de început al arclui, apoi este selectat și celălalt capăt;
- **5 (Polygon)**: Desenează un poligon cu o forma arbitrară;
- **6 (Circle)**: Desenează un cerc;
- **7 (Rect)**: Desenează un dreptunghi;

Câteva exemple de capsule:



După ce avem forma de bază a componentei, e necesar să definim și pinii piesei. Este recomandat ca aceștia să fie numiți așa cum îi găsim în datasheet. De asemenea, symbol-ul trebuie gândit astfel încât originiea componentei („+” prezent în figura de mai jos) să fie în centrul piesei. ! Chiar dacă din desenul pe care îl faceți se pot observa conexiunile componentei, trebuie utilizați și pini. !
După finalizarea desenului, putem adăuga directivele ***>NAME*** și ***>VALUE*** (ca text) pe layer-ele *“95 Names”*, respectiv *“96 Values”* pentru a ni se afișa automat în schema pe care o desenăm un nume și o valoare pentru

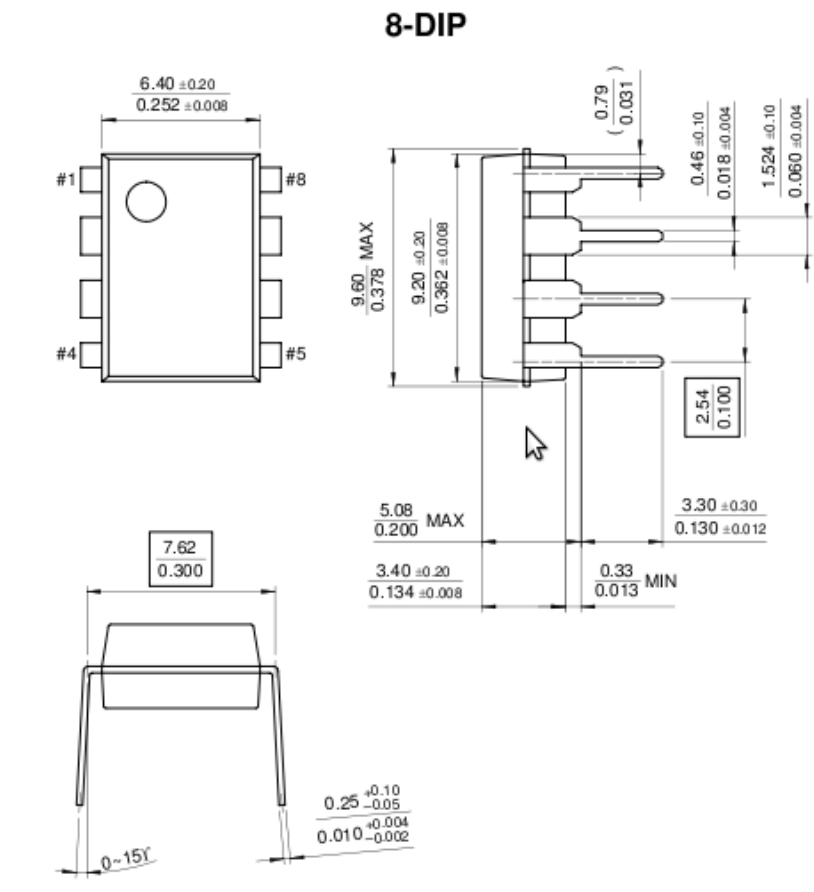
componentă (ex: rezistenta: valoarea si numele R, condensator: valoarea si numele C).



Nu schimbați dimensiunea gridului! Altfel nu veți mai putea să conectați firele la pini în modul de editare al schematicului. Pasul standard pentru un grid este de 0.1 inch.

Package

După ce avem simbolul gata, trebuie să creăm și modul în care va apărea piesa pe board, acesta fiind rolul package-ului. Pentru acest lucru, este necesar să știm dimensiunea și forma pinilor, precum și amplasamentul lor pe placă (distanța dintre ei). Toate aceste date formează ceea ce se numește „footprint-ul” piesei și poate fi regăsit, de obicei, în datasheetul piesei respective (vezi un exemplu în figura de mai jos).



Veți observa că pentru fiecare dimensiune apar câte două valori: cea de sus reprezintă valoarea în milimetri, iar cea de jos e în inch. De asemenea, in unele datasheet-uri, vom observa ca putem avea doua valori (ex: valori pentru capsula) sau de forma valoare +- eroare. Acestea reprezinta dimensiunea intre care se poate incadra respectiva capsula, avand in vedere eroare de fabricatie.

Vor exista cazuri când va fi nevoie să modificați dimensiunea gridului pentru a putea plasa pad-urile conform cu dimensiunile footprint-ului. Puteți face acest lucru din tool-ul Grid, după cum am învățat în laboratorul trecut.

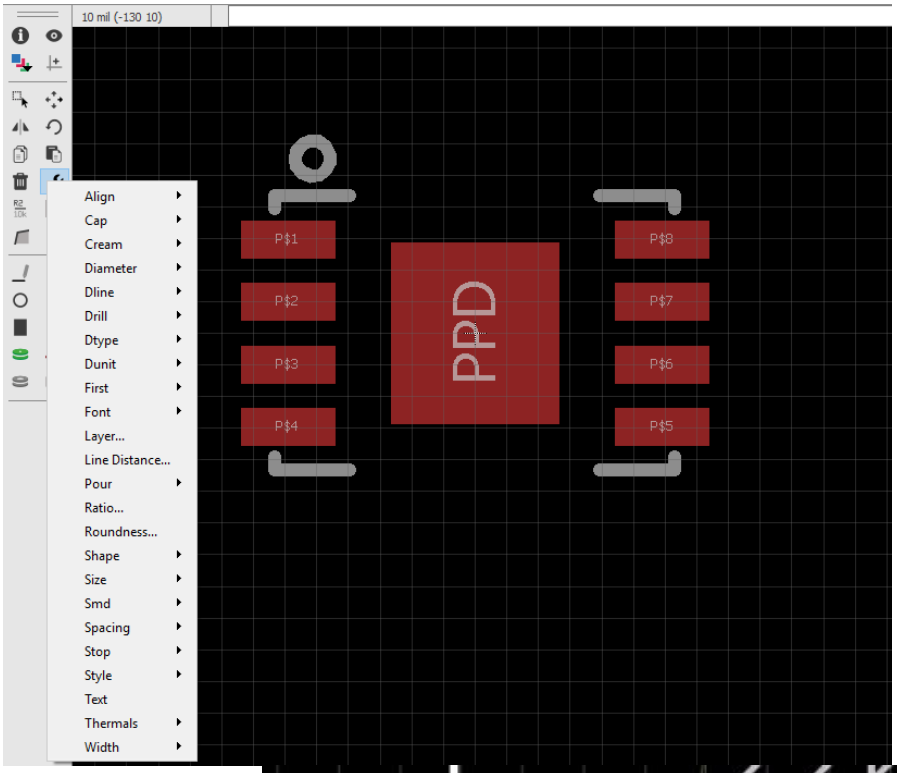
Componentele pe care le putem adăuga în modul Package sunt:

- Pad-uri:
- SMD;
- DIL;
- Hole;
- Restul componentelor fac parte din silkscreen.

Silkscreen reprezinta stratul de vopsea ce se regaseste pe suprafata PCB-ului, utilizat pentru a identifica componente, puncte de testare, părți ale PCB-ului, simboluri de avertizare, logo-uri etc.

Înainte de a adăuga pad-urile, trebuie să ne asigurăm că au dimensiunea potrivită. După ce vedem în datasheet dimensiunea recomandată, o putem selecta din bara de sus de opțiuni (de asemenea, și forma):

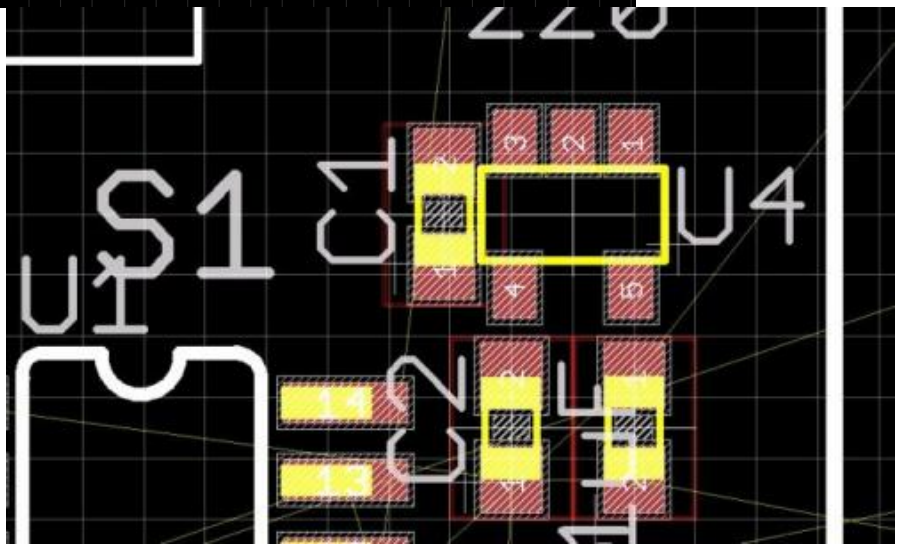
Daca totuși trebuie modificate apoi, puteți oricând schimba diferite opțiuni din sidebar, folosind tool-ul „Change” ca în figura de mai jos:



Orice meniu de selecție a dimensiunilor are și o opțiune „…” care vă permite să adăugați dimensiuni custom, în cazul în care nu ați găsit dimensiunea care vă trebuie printre exemplele date.

După adăugarea tuturor pad-urilor și plasarea pad-urilor pe pozițiile corespunzătoare, asigurați-vă că aveți piesa centrată (similar cum am făcut pentru schematic,re-poziționarea tuturor elementelor pentru a încadra „+”-ul). Apoi trebuie să numiți pad-urile conform cu datasheet-ul.

Piesa este aproape gata! Trebuie acum să îi adăugăm și dimensiunile fizice ale capsulei. Aceste dimensiuni se adaugă pentru referință, pentru a nu face greșeala de a pune pe placă două piese care sunt prea apropiate, lucru care ar face lipirea lor aproape imposibilă (vezi figura de mai jos, liniile galbene reprezintă capsulele pieselor. Acestea se afla in layerul 20 Dimension).



Pentru a desena și capsula, puteți selecta tool-ul Wire, folosind Layer-ul 51 tDocu. Folosiți dimensiunile din datasheet!

Opțional, pe lângă capsulă, puteți realiza și silkscreen-ul pe Layer-ul 21 tPlace, care reprezintă liniile ajutătoare pentru plasarea piesei pe PCB.

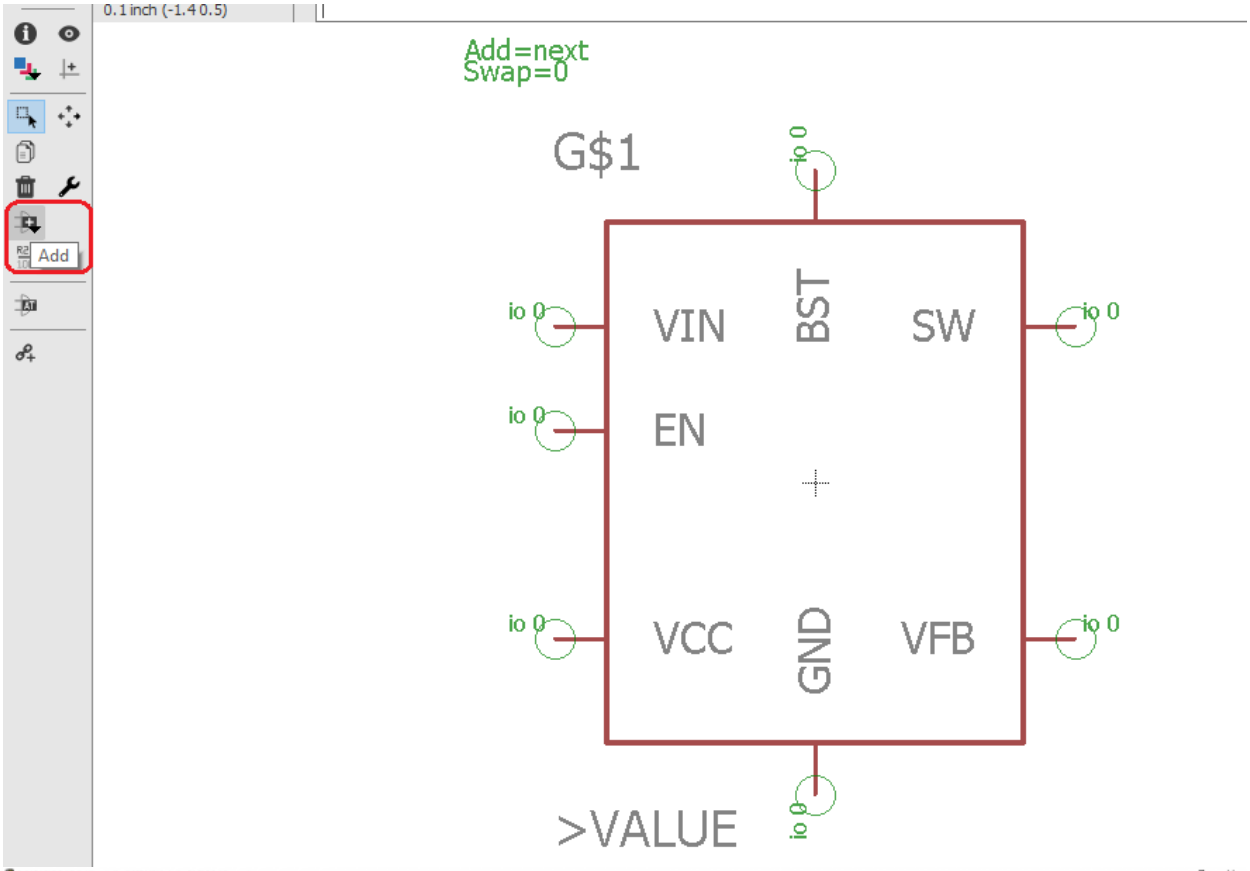
Device

Acum că avem Symbol-ul și Package-ul, trebuie să îi spunem lui Eagle ce pini din Symbol trebuie asociati cu ce pini din Package. Pentru început, treceți în modul de editare Device, folosind butoanele din toolbar-ul din partea de sus.

Apoi, folosiți butonul „Add” pentru a adăuga simbolul nou-creat:

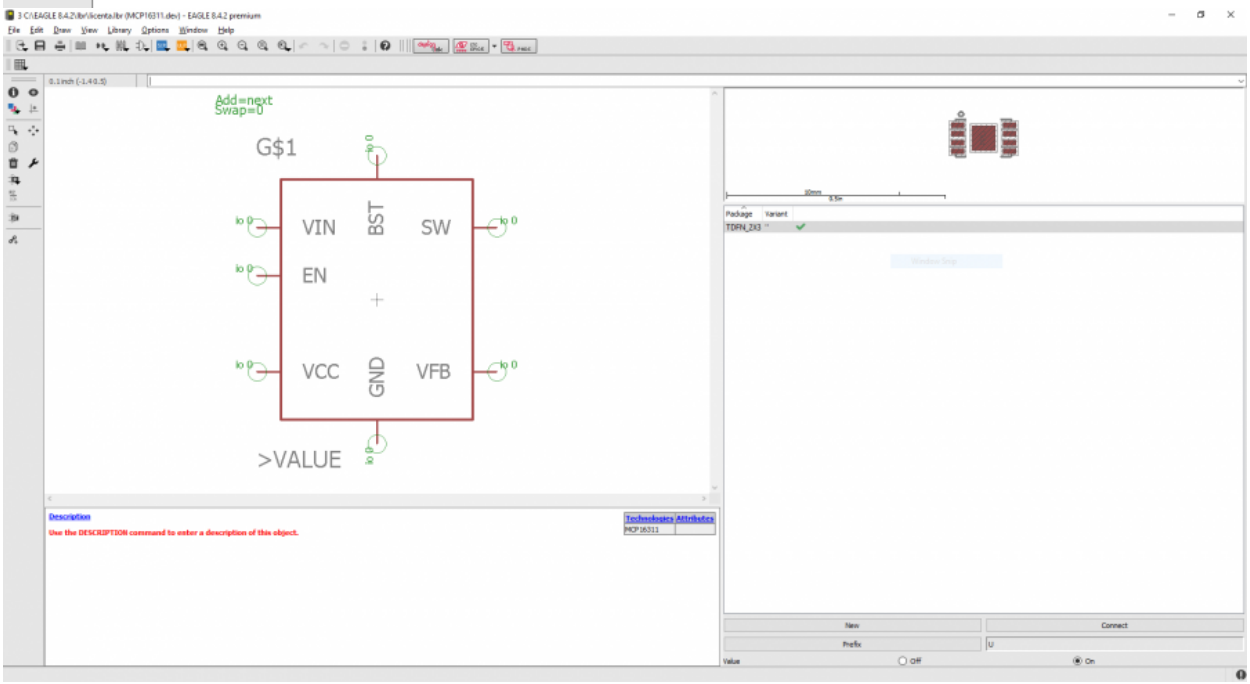
Simbolul va apărea centrat, iar din partea dreaptă a ferestrei veți putea adăuga Package-ul, folosind butonul New (dreapta jos) → Add local Package:

În figura de mai sus, semnul de exclamare din partea dreapta jos indică faptul că pinii nu au fost asignați încă. Apăsați butonul *Connect* pentru a deschide fereastra de asignare a pinilor între Symbol și Package. Folosiți butonul **Connect** pentru a conecta un Pin din Symbol cu un Pad din Package sau **Append** pentru a conecta mai multe Pad-uri la un Pin (ex: unele circuite integrate pot avea

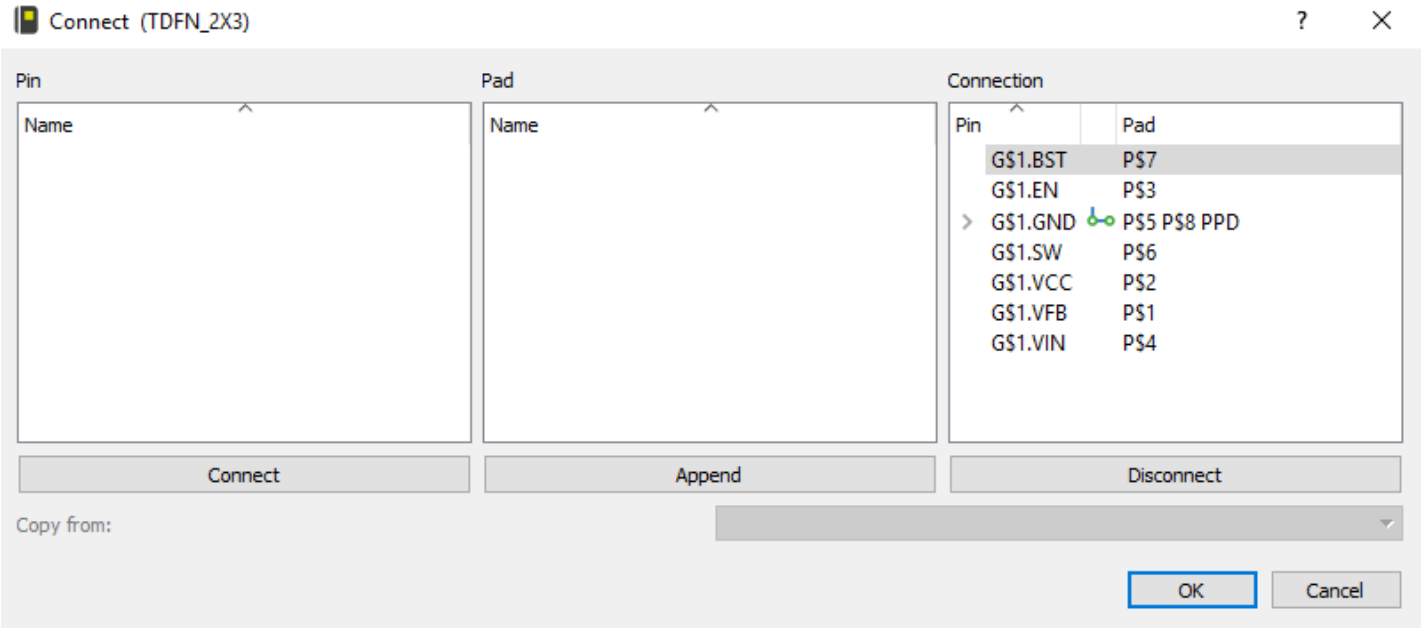


mai multe Pad-uri de Ground):

Asignați pinii conform cu



datasheetul.



Exerciții

1. Realizați componenta LM741 [http://www.ti.com/lit/ds/symlink/lm741.pdf] într-o bibliotecă nouă. Trebuie să implementați:
    a. Symbol-ul
    b. Package-ul (inclusiv: dimensiunea capsulei!)
    c. Și să le conectați într-un Device.
    - Hint: Aveți grijă ca toți pinii să aibă un pad asociat conectat.
    - Hint: Aveți grijă să lucrați cu unitățile de măsură corecte! Implicit EAGLE folosește MIL(thousands of an inch) pentru grid-ul din footprint/board view. Dacă vreți să lucrați cu milimetri, va trebui să schimbați unitatea implicită cu mm folosind View→Grid…
2. Realizați rezistența în capsula SMD 1206 disponibilă aici [https://www.vishay.com/docs/28705/mcx0x0xpro.pdf], respectând pașii de la exercițiul anterior.

Resurse

- Tutorial AutoDesk de creare a unei componente noi [https://www.autodesk.com/products/eagle/blog/library-basics-part-1-creating-first-package-autodesk-eagle]
- Tutorial SparkFun de creare a unei componente noi [https://www.sparkfun.com/tutorials/110]
- Biblioteca RCL [https://drive.google.com/open?id=1iUwAfbEe-sa6rf6RqSVRMC9LD4UvC0Ld]

icalc/laboratoare/laborator-02.txt · Last modified: 2022/03/01 12:25 by giorgiana.vlasceanu

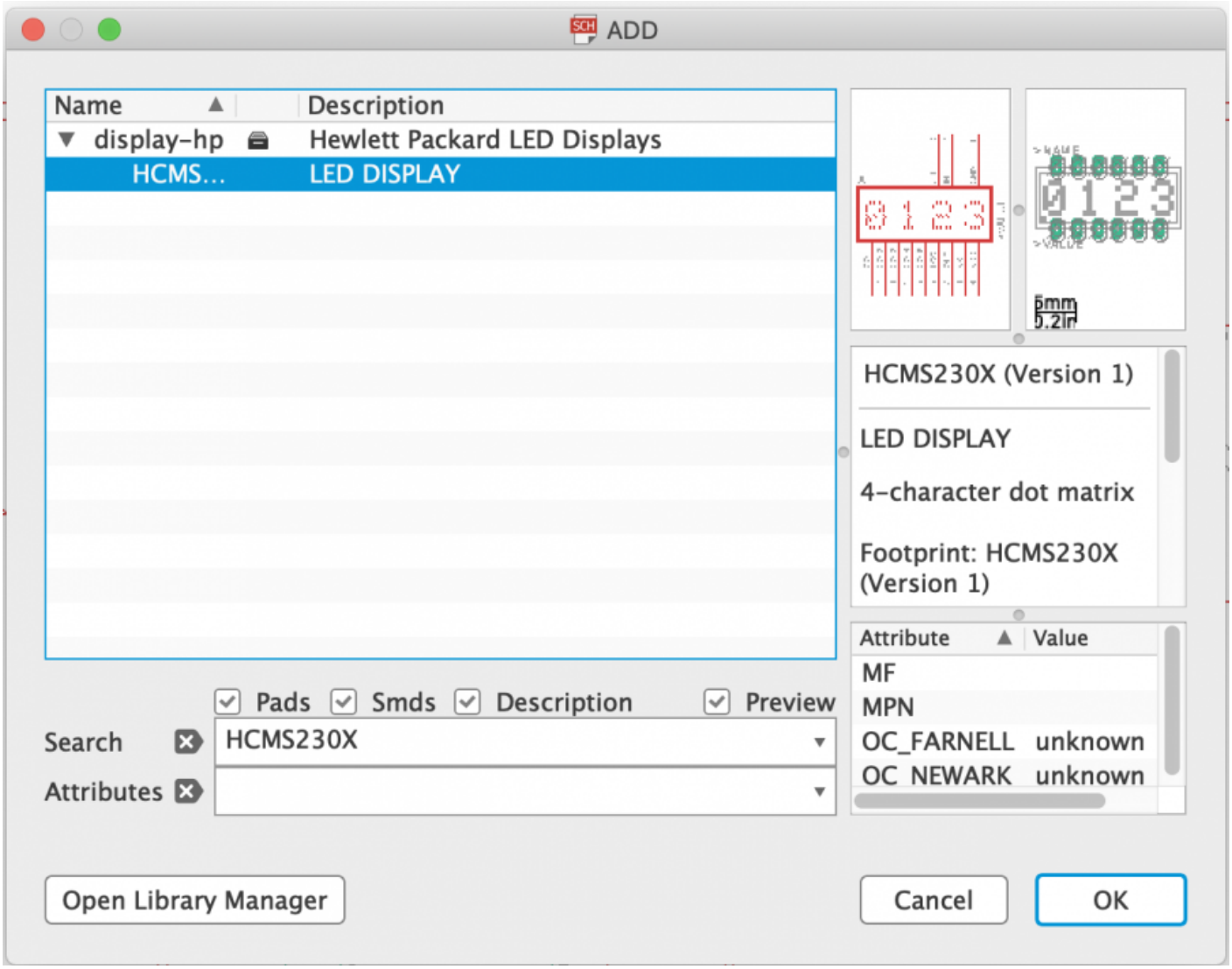
Laborator 03 - Eagle 103

Tips and tricks

Cautare componente

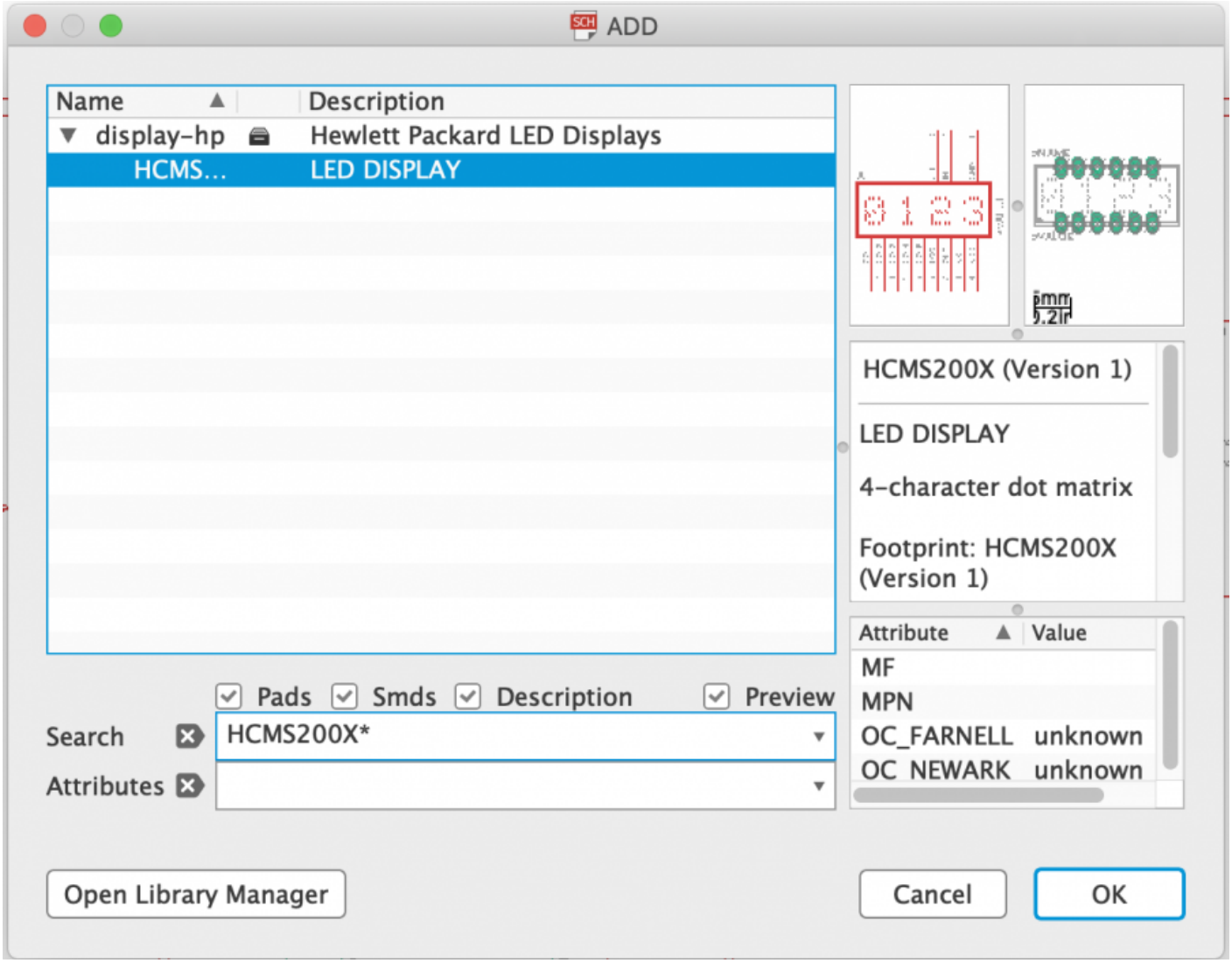
Pentru a cauta o componenta in Eagle se pot folosi urmatoarele metode:

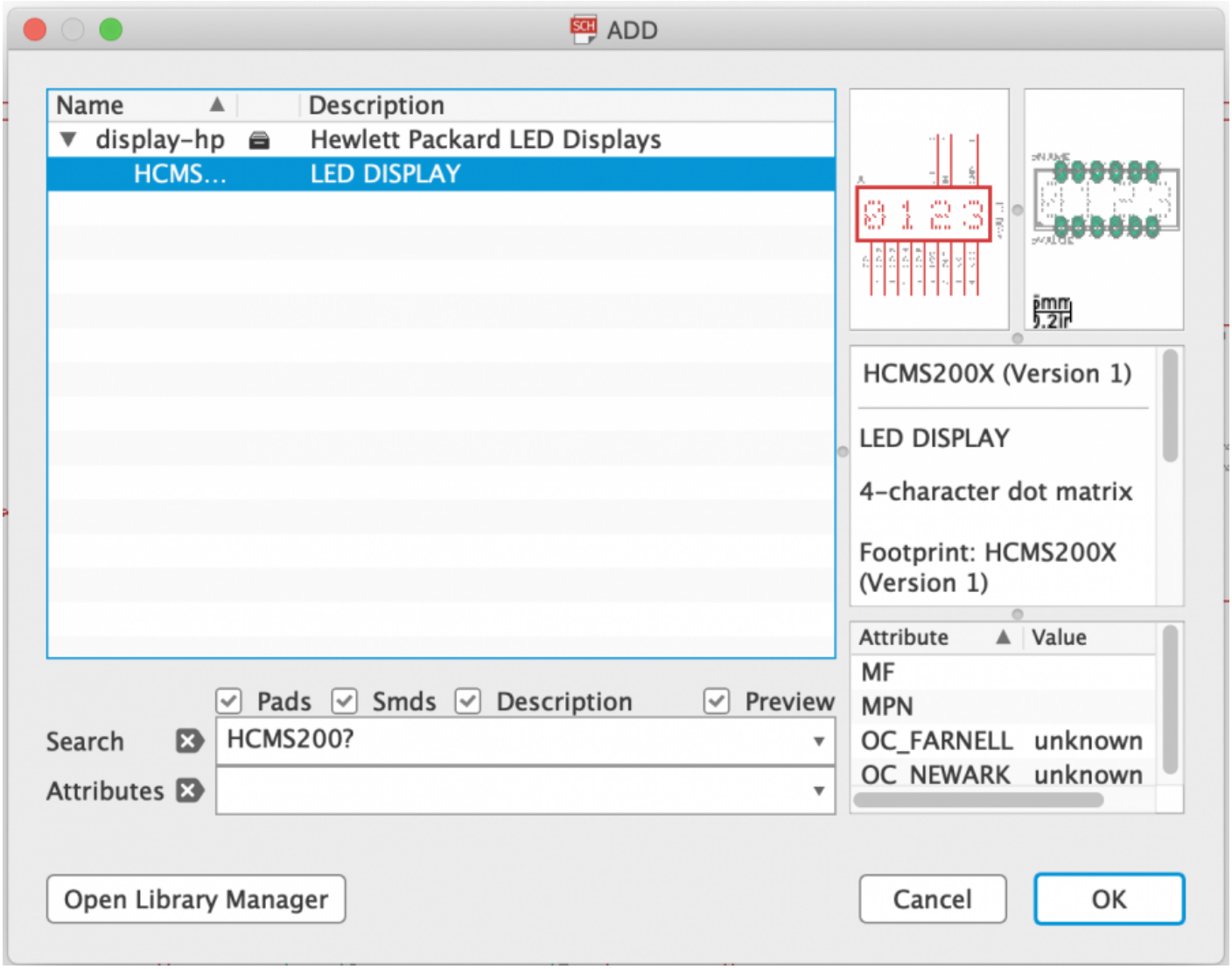
- **1.** Cautarea componentei dupa numele ei exact (ex: HCMS230X).



- **2.** Folosirea caracterului “*” inainte sau dupa o portiune de nume. Acesta are rolul de a inocui una sau mai multe litere (ex: *MS200X sau HCMS2*).

- **3.** Folosirea caracterului ”?”. Acesta are rolul de a inlocui o litera (ex: HCMS200?)


ADD
Name
Description
display-hp
Hewlett Packard LED Displays
HCMS...
LED DISPLAY
HCMS200X (Version 1)
LED DISPLAY
4-character dot matrix
Footprint: HCMS200X (Version 1)
Attribute
Value
MF
MPN
OC_FARNELL unknown
OC_NEWARK unknown
Pads
Smds
Description
Preview
Search
HCMS200X*
Attributes
Open Library Manager
Cancel
OK



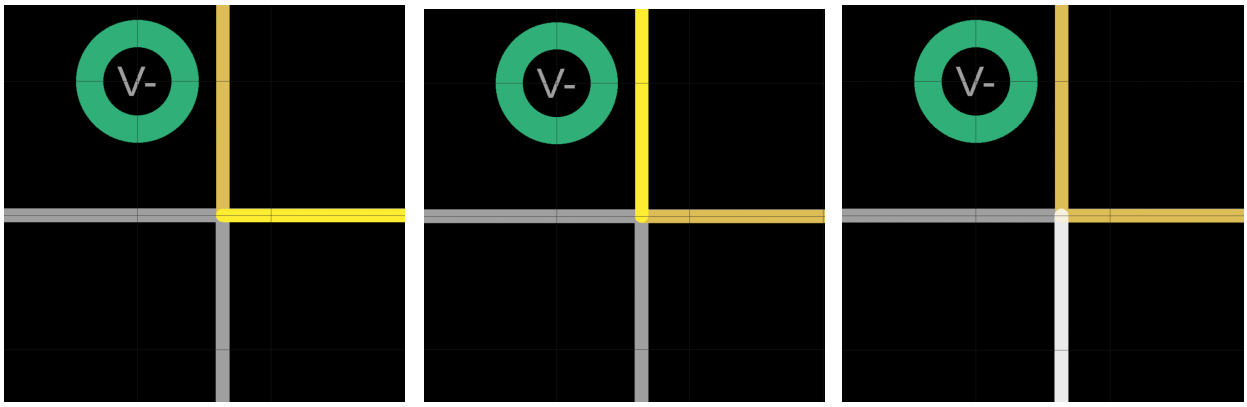
Selectare componente concurente

In laboratoarele trecute am intampinat situatii in care existau mai multe elemente ce aveau originea in acelasi punct. Acest lucru face dificila selectarea componentei dorite. In continuare vom lua exemplul in care dorim sa stergem un element ce se afla intr-un punct. Vom selecta unealta de stergere si apoi vom apasa click stanga pe punctul de concurenta. Utilizand click dreapta putem naviga prin toate elementele ce se afla in acel punct. In momentul in care avem selectat elementul dorim putem efectua actiunea dorita folosind click dreapta. In cazul de faza ne va sterge elementul selectat.

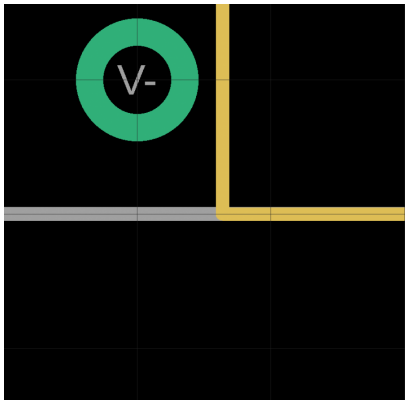
- **Pas1:** Selectarea uneltei de stergere.



- **Pas2:** Navigarea prin elemente.

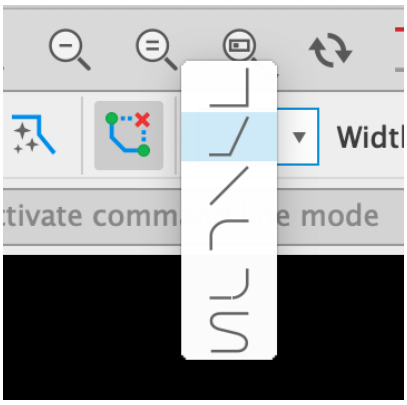


- **Pas3:** Stergerea elementului selectat.

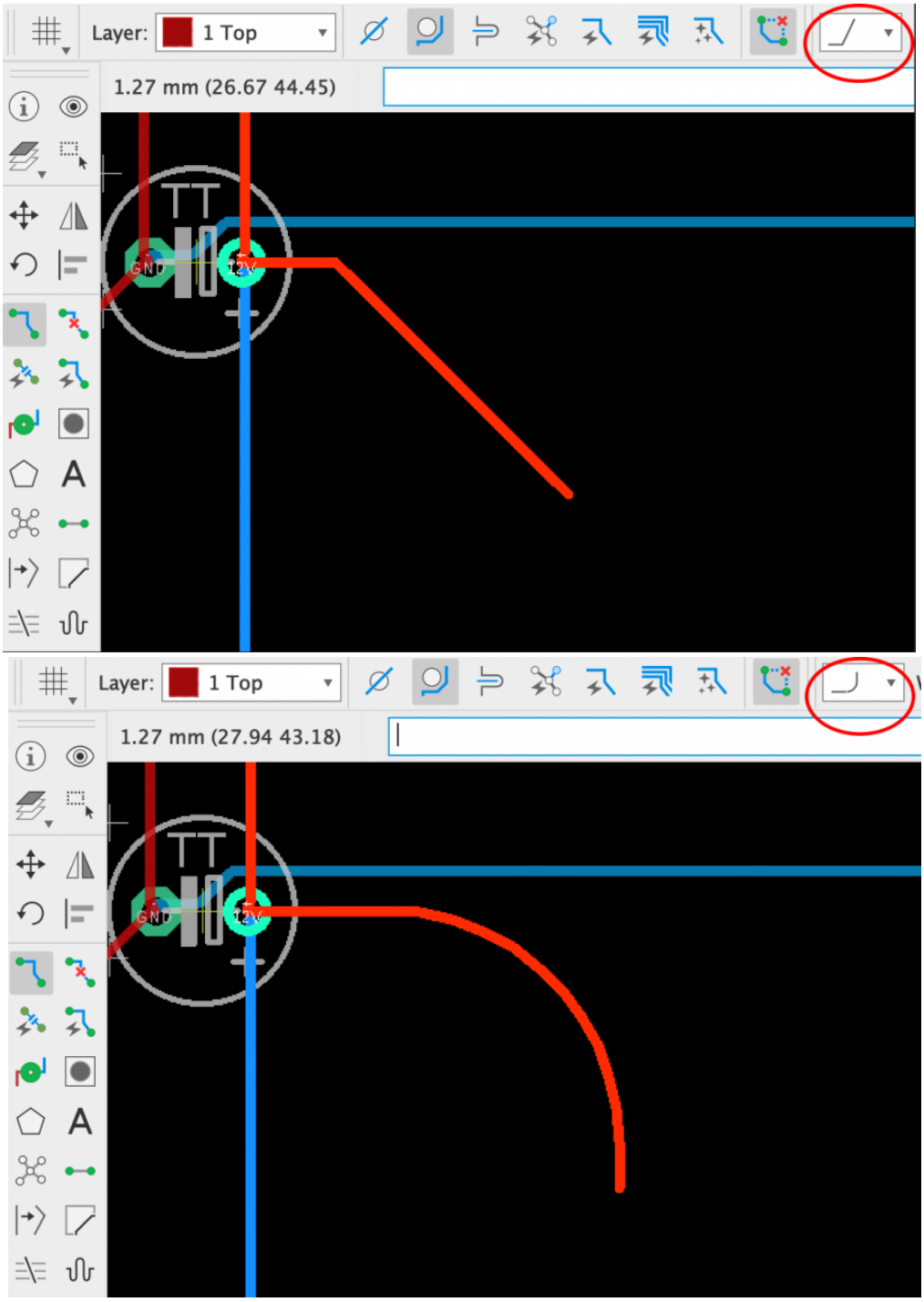


Selectarea modului de rutare a firelor

Dupa selectarea uneltei de rutare, in meniul de optiuni, avem tipurile pe care le putem alege.

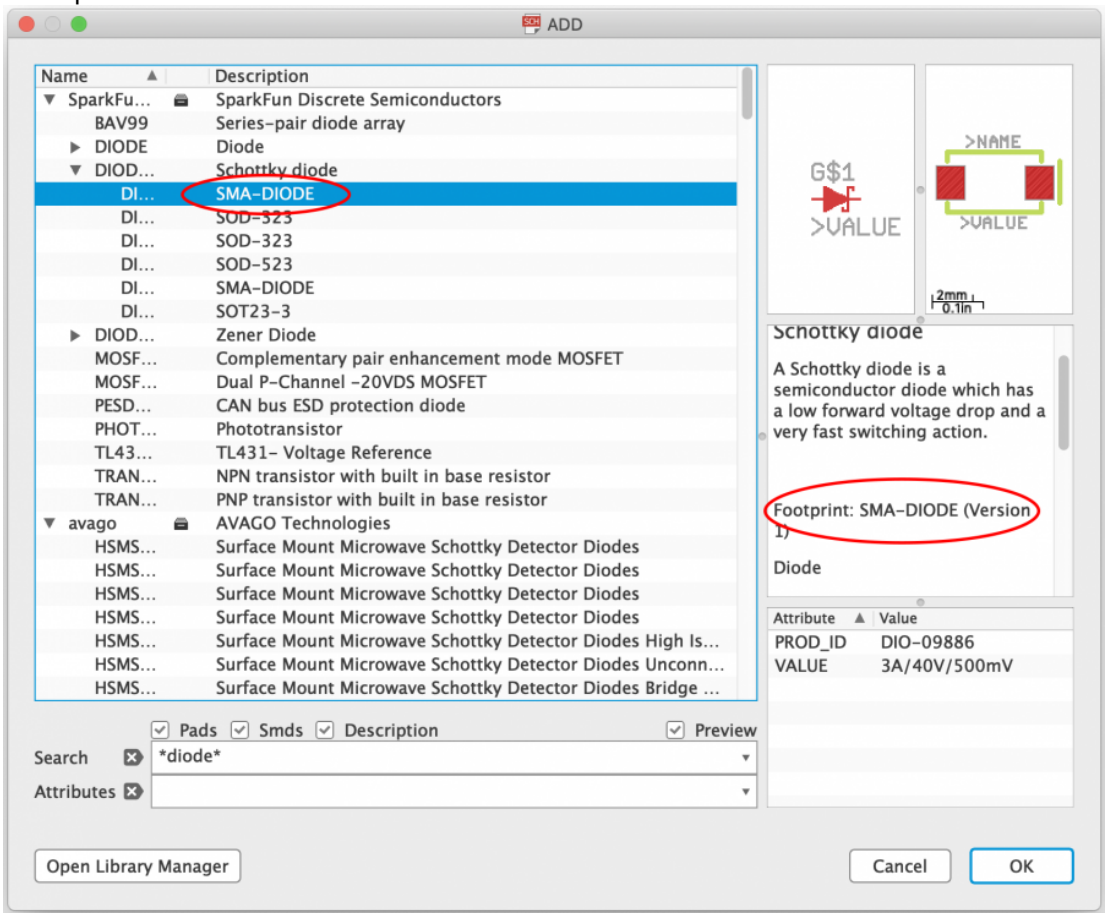


Prin acest meniu putem naviga si folosind “click dreapta”, atunci cand avem selectat utilitarul de rutare.



Cautarea unei componente si a capsulei corespunzatoare

Pentru a diferentia intre SMD si THT putem sa privim footprintul componentei. Daca observam pad-uri atunci este THT, altfel este SMD. O alta metoda este sa citim descrierea componentei. Vom putea observa pachetul ori in coloana description ori in descrierea detaliata a acesteia.



Exerciții

Folosind schema de aici, realizați schematicul în EAGLE si apoi PCB-ul(design-ul placii fizice). Veti folosi atat componente in capsule THT(Through-Hole Technology), cât și SMD(Surface Mount Device).

- Hint: Folosiți Wire (Style: longdash) pe layer-ul 92 Busses pentru delimitarea între diferite părți ale PCB-ului;
- Hint: Firele denumite la fel sunt unite la nivel electronic, chiar dacă nu apar conectate în schematic;
- Pentru design-ul unor părți ale PCB-ului, puteti folosi si functia de autorouter (NU traseele de semnal și putere).

Recomandări realizare schemă:

- Instalare biblioteci necesare: Sparkfun-IC-Microcontroller, SparkFun-DiscreteSemi, SparkFun-Resistors, SparkFun-Capacitors, SparkFun-Switches
- Microcontroller-ul poate fi găsit în biblioteca Sparkfun-IC-Microcontroller;
- Tranzistorii pot fi găsiți în bibliotecile SparkFun/SparkFun-DiscreteSemi, căutând *npn*, *nmos* sau *mosfet*;
- Diodele pot fi găsite în biblioteca SparkFun-DiscreteSemi, căutând *diode*;
- Potențiometrul poate fi găsit în biblioteca SparkFun-Resistors, căutând *trimpot*;
- Switch-ul S1 îl puteți găsi în biblioteca SparkFun-Switches, căutând *tactile*;
- JP1 și JP2 reprezintă câte un pinheader și pot fi găsiți căutând *pinh*;
- Cristalul de quartz poate fi găsit in biblioteca adafruit, căutând *crystal*.
- Respectarea valorilor componentelor (ex: rezistente, condensatoare)
- De obicei numele capsulei apare scris in dreptul coloanei description

Exemplu:

Name	Description
MOSFET	TO220V

Constrângeri capsule:

- Tranzistorii Q5 și Q6 trebuie să fie în capsulă SOT23-3;
- Tranzistorii MOSFET de putere (Q2, Q3, Q4 și Q8) trebuie să fie în capsulă TO220;
- Quartz-ul Q1 trebuie să fie în capsulă HC49/S;
- Condensatorii C1, C2, C3, C4 și C5 trebuie să fie SMD, în capsulă 0805;
- Condensatorii C6 și C7 trebuie să fie polarizați, în capsulă THT;
- Microcontroller-ul trebuie să fie în varianta THT;
- Diodele D1, D2, D3 și D4 trebuie să fie diode Schottky, în capsulă SMA.

Reguli de good practice în realizarea PCB:

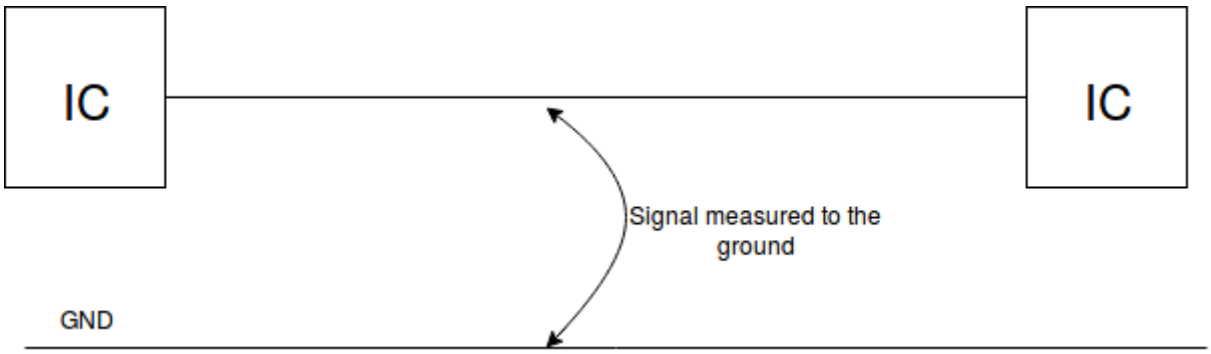
- Traseele de putere (cele pentru alimentarea punții H și cele pentru alimentarea motorului de la puntea H) este recomandat să fie cât mai scurte și destul de groase (minim 2.54 mm grosime);
- Traseele de semnal (PWM_1 și PWM_2) trebuie să fie cât mai scurte și să nu fie plasate în zona traseelor de putere;
- Condensatorii de decuplare (C3, C4 și C5) trebuie plasați cât mai aproape de pinii microcontroller-ului (fiecare condensator la câte un pin de alimentare/referință);
- Traseele nu trebuie să aibă unghiuri drepte;
- Cristalul de quartz trebuie pus foarte apropiat de microcontroller, la fel și cei doi condensatori;
- Dacă placa este realizată dublu strat, se vor evita vias-uri la traseele de putere.

Se poate folosi parțial și Autorouter dacă se îndeplinesc regulile de good practice; daca este nevoie, placa poate fi realizată pe două straturi.

icalc/laboratoare/laborator-03.txt · Last modified: 2022/03/30 18:37 by liviu.mitruta

Laborator 04 - Eagle 104 - Semnale diferențiale

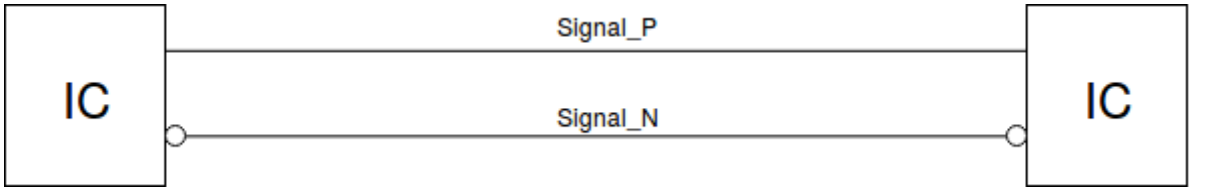
Pana acum circuitele pe care le-am proiectat au transmis semnale care aveau ca punct de referință masa circuitului (single-ended signaling).



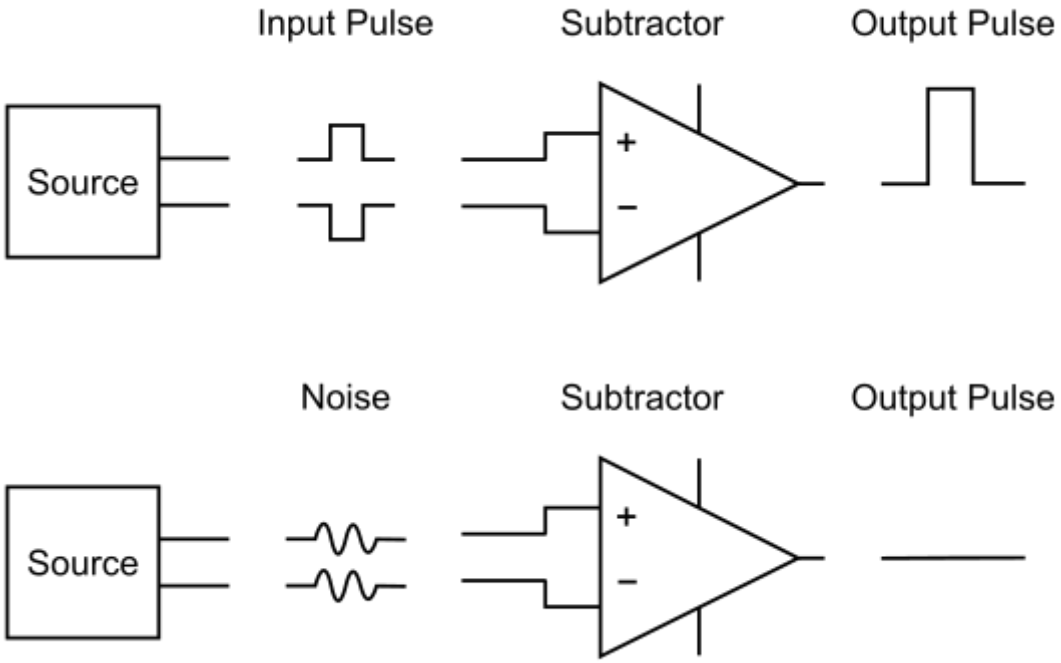
Daca dorim sa transmitem semnale cu viteza mare, atunci zgomotul va fi un factor important de luat in considerare:

- In primul rând zgomotul electro-magnetic afectează toate traseele de pe un PCB
- In al doilea rând zgomotul din masa; Deoarece masa este comuna cu întreaga logica de pe PCB, zgomotul poate fi indus de oricare dintre componente. Astfel valoarea de referință poate fi ridicată sau coborâtă, ducând la citiri eronate.

In acest laborator ne vom ocupa de semnale diferențiale. Spre deosebire de transmiterea datelor printr-un semnal single-ended in cazul semnalelor diferențiale datele sunt transmise prin doua semnale complementare. Circuitul care va interpreta datele nu va mai citi diferența de tensiune intre intrare și masa, ci va reacționa la diferența dintre cele doua semnale.



Datorita faptului ca in circuitul diferențial impedanțele de transmitere si recepție sunt egale, interferentele electromagnetice tind sa afecteze in mod similar ambele fire, inducand acelasi zgomot pe ambele fire. Deoarece Informatia relevanta este obtinuta prin diferenta celor doua semnale, zgomotul va fi redus si astfel efectul lui asupra circuitului va fi minim.

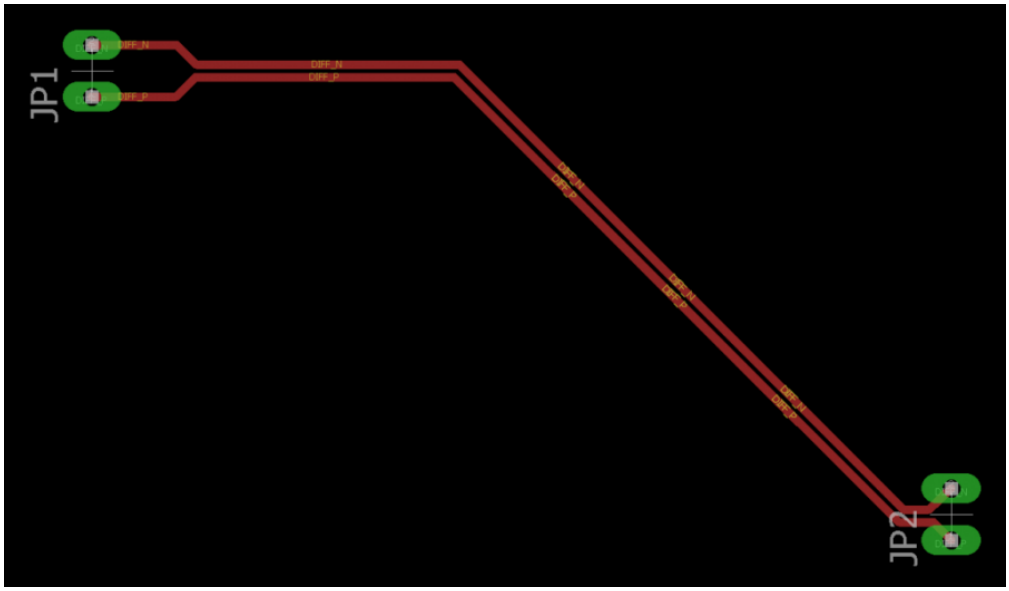


Protocoale care implementează transmisia datelor folosind semnale diferentiale:

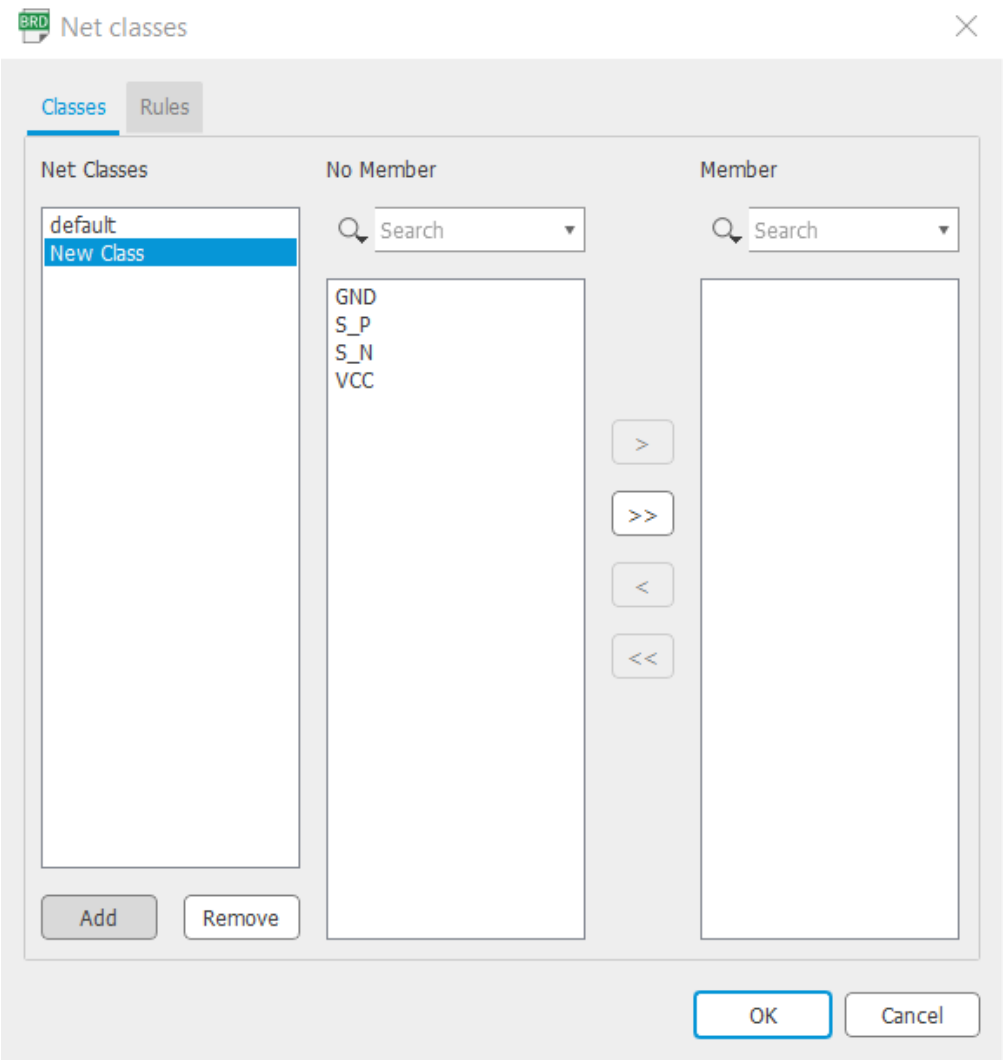
- SATA
- HDMI
- USB
- PCI Express
- Ethernet over twisted pairs

Rutarea semnalelor diferențiale

Pentru a putea ruta o pereche de semnale diferențiale este necesar ca semnalele sa fie denumite corespunzător. Primul semnal trebuie sa aibă numele de forma nume_P, iar al doilea difera doar prin sufix: _N. In exemplul de mai jos am notat cu DIFF_P si DIFF_N cele doua semnale diferențiale.



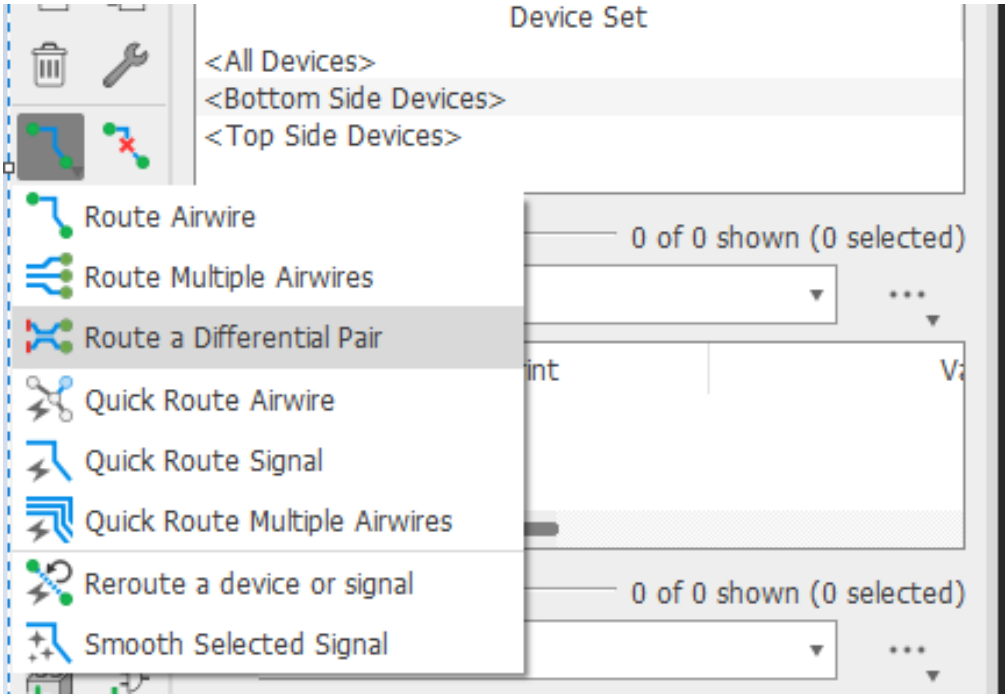
In .brd, vom defini o noua clasa de semnale, mergand in meniul Edit → Net Classes → Add.



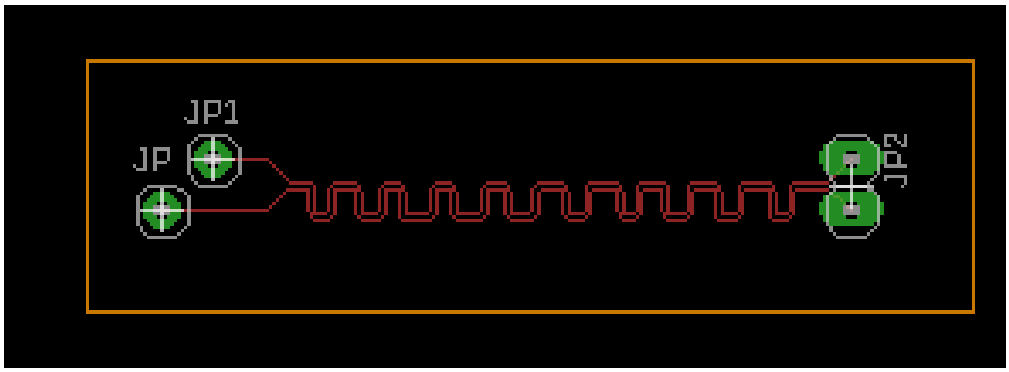
In aceasta fereastra se adauga perechea de semnale diferentiale in noua clasa.

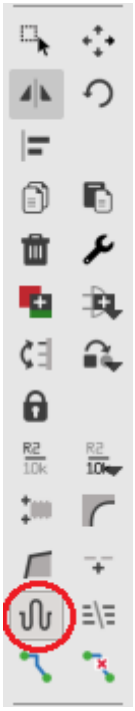
Pentru a fi recunoscute ca semnale diferentiale de Eagle, este necesar ca numele semnalelor sa aibe sufixele '_P' pentru pozitiv, respectiv '_N' pentru negativ.

Rutarea semnalelor diferentiale se face prin selectarea modului **Route a Differential Pair** a instrumentului **Route**.



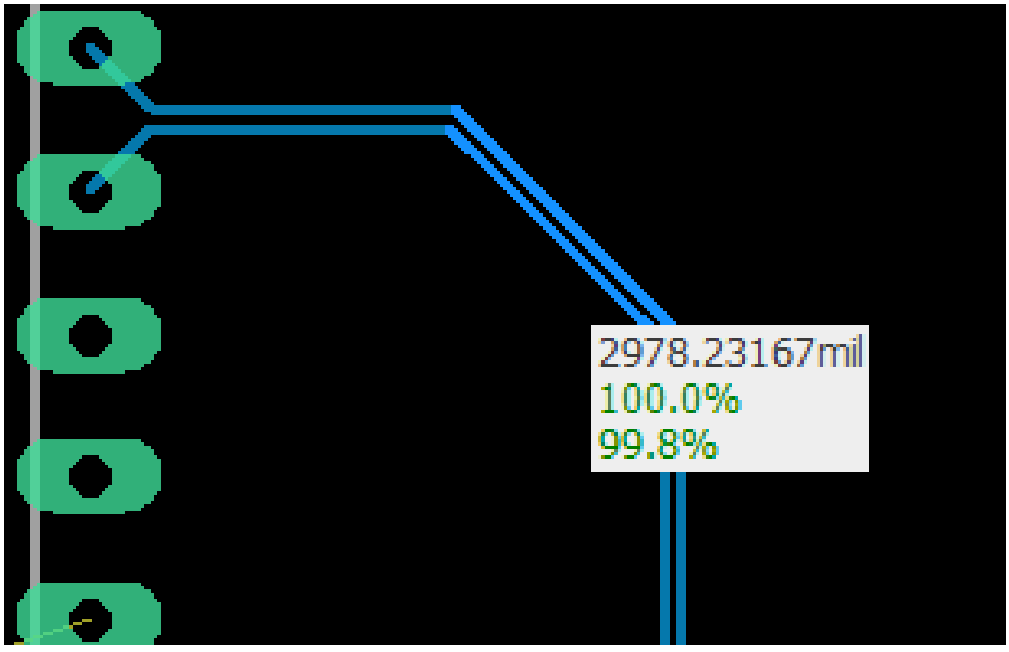
O constrangere importanta este faptul ca cele doua fire trebuie sa aiba lungime egala, pentru acest lucru se foloseste rutarea in serpentine pentru a aduce lungimea traseelor sa fie cat mai apropiata.





Functia din eagle se numeste Meander si o puteti gasi astfel:

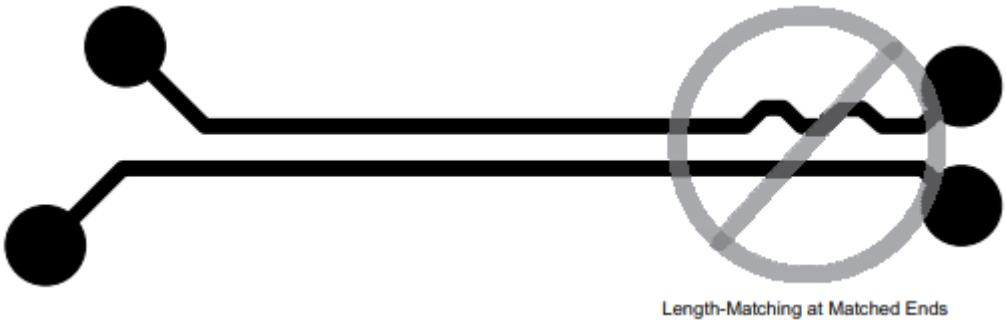
Cand selectam o portiune de traseu cu functia Meander, va fi afisata lungimea comparativa a celor doua trasee.

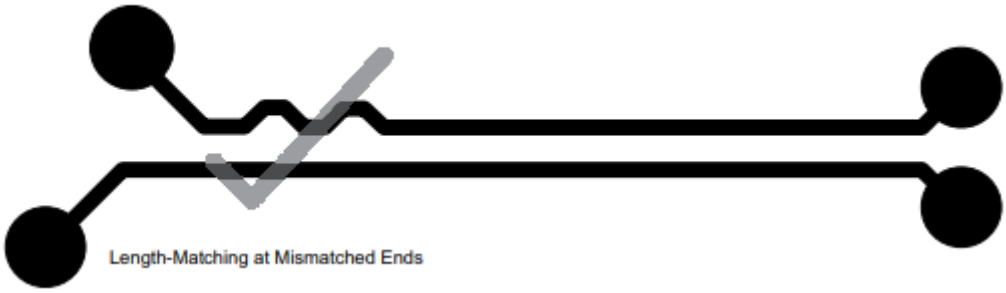


Constrangeri si Recomandari

Constrangeri de lungime

Dupa cum am am menționat anterior, lungimea ambelor fire trebuie sa fie cat mai apropiata. Este “good-practice” ca serpentinele sa fie cat mai apropiate de capatul in care pad-urile/via-urile nu sunt aliniate corespunzător.



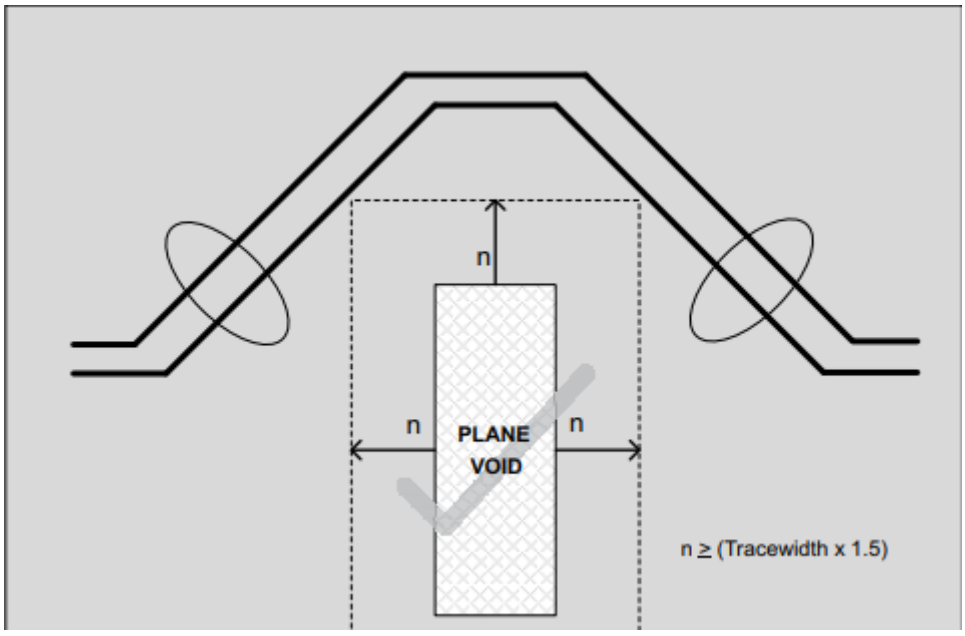


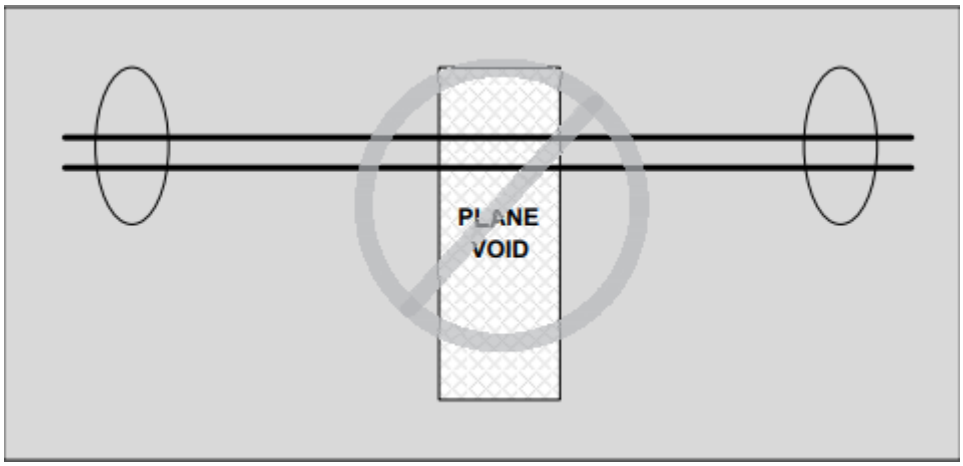
Situarea fata de planul de referință

Firele pe care circula semnale de mare viteza trebuie sa fie rutate mereu pe un plan de GND. Rutarea peste un plan vid poate cauza:

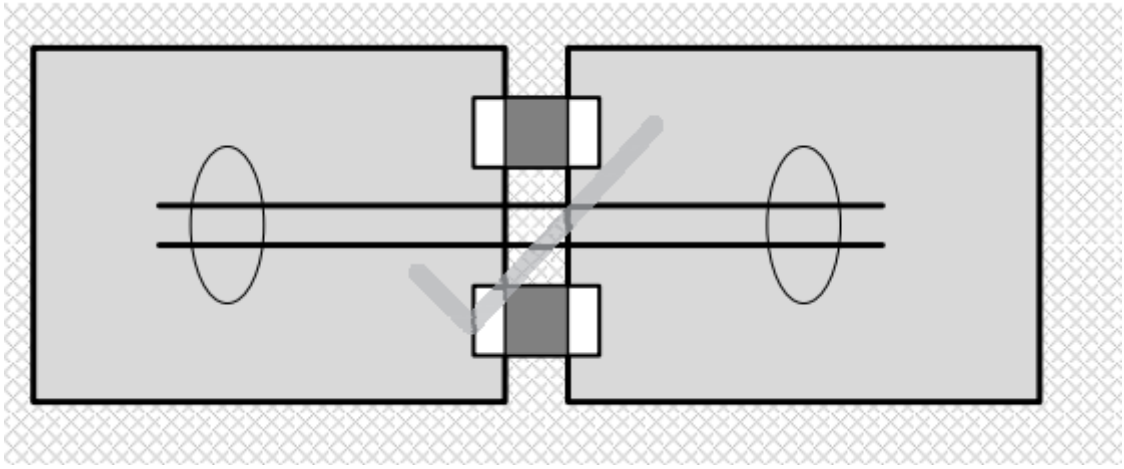
- Întârzieri in propagarea semnalelor
- Interferente
- Degradarea integrității semnalelor

Exemplu:

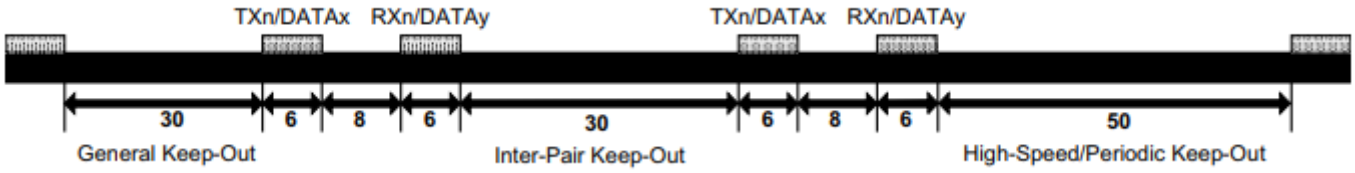


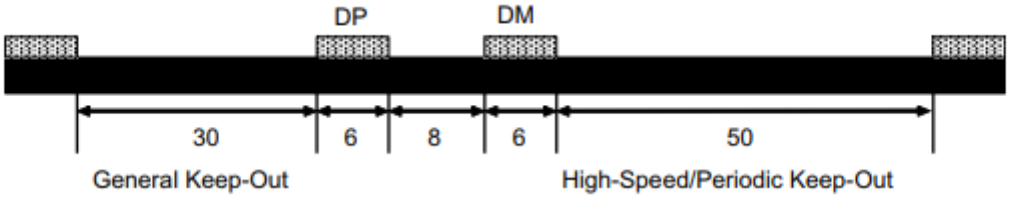


Daca nu se poate evita trecerea peste un plan vid, înseamnă ca avem doua planuri de masa distincte. Este recomandat ca acestea sa fie conectate prin doua condensatoare de capacitate 1 microF sau mai puțin, plasate cat mai aproape de firele de semnal.



Distanta intre fire Este recomandat ca distanta intre perechile de fire sa fie de 5xW, unde W este latimea firului. Este recomandat ca semnalele periodice sa fie izolate folosind o distanta de cel putin 8xW.





Diverse reguli:

- Este recomandat sa nu se ruteze fire pe care circula semnale de inalta frecventa pe sub sau langa cristale de quartz, generatoare de ceas, gauri pentru montare, dispozitive magnetice, regulatoare de tensiune sau IC-uri care folosesc semnalul de ceas.
- Este recomandat ca firele pe care circula semnale diferentiale sa fie rutate la cel putini 90mil de marginea planului de referinta,

- Este recomandat sa se maximizeze distanta intre perechile de fire de transmisie atunci cand este posibil.
- Perechile de fire trebuie trasate paralel, singurul punct in care este permisa spargerea simetriei este inainte de a se conecta la capsula de siliciu a unui circuit integrat. Distanta recomandata pentru spargere este cu 0.25inch inainte de capsula acesteia.

Planul de masa

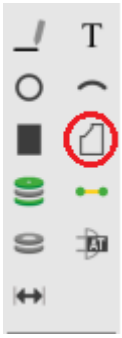
Planul de masa de pe un PCB este o zona larga acoperita cu un strat de cupru care este conectata la punctele de masa ale circuitului. Acesta este realizat de regula cat mai mare, acoperind aproape intreaga suprafata a PCB-ului care exceptand zonele pe unde circula traseele.

Planul de masa are urmatoarele avantaje:

- In primul rand aranjarea componentelor pe PCB devine mult mai usoara deoarece accesul pinilor la un punct de masa este relativ facil
- Reduce zgomotul si interferentele cauzate de buclele de masa.

Cum se poate realiza un plan de masa in Eagle?

- Se selecteaza functia **Polygon**



- Se delimiteaza zona in care dorim sa fie planul de masa
- Se redenumeste poligonul folosind numele **GND**
- Se foloseste functia **Ratsnests**



Tip: pentru a elimina ilustrarea planului de masa (umplerea chenarului), puteti folosi comanda `ripup @;`.

Exerciții

- Realizati schema de aici.
  - Convertorul USB-UART FT232RL poate fi gasit in biblioteca Sparkfun-IC-Comms
  - Portul USB poate fi gasit in biblioteca OPL_Connector si cautand *DIP-USB*
  - Condensatorii C1 si C4 trebuie să fie SMD, în capsulă 0805;
  - Condensatorii C2 și C3 trebuie să fie polarizați, în capsulă THT
  - Ferrite bead-ul il gasiti in biblioteca Sparkfun-Coils sau cautand *ferrite_bead*
- Realizati PCB-ul (Se cere respectarea constrangerilor si recomandarilor facute in acest laborator; Nu este permisa folosirea functie **Autorouter**)
- Realizati planul de masa

icalc/laboratoare/laborator-04.txt · Last modified: 2022/05/24 00:59 by liviu.mitruta

Laborator 05 - Git și refactorizare

Obiectiv

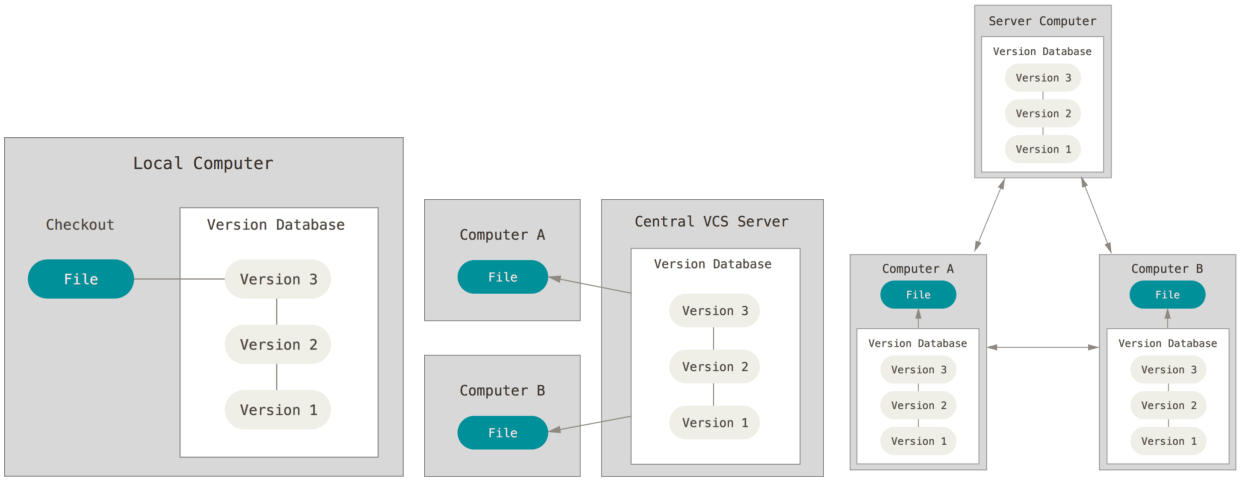
În prima secțiune scopul este familiarizarea cu mecanisme de versionare a codului în special Git/GitHub. În a doua secțiunea se prezintă bune practici pentru a menține codul curat.

Version Control

Un sistem pentru controlul versiuni este un sistem care înregistrează modificările suferite de un fișier sau un grup de fișiere în decursul timpului pentru a facilita revenirea la o versiune specifică ulterior.

Tipuri de sisteme pentru controlul versiunii

Exista 3 tipuri de sisteme pentru controlul versiunii:



Sisteme locale pentru controlul versiunii

Cele mai vechi VCS-uri (VCS – Version Control System) au presupus că toți dezvoltatorii lucrau în propriile directoare pe un singur sistem de fișiere partajat. Repository-ul a fost păstrat într-un director separat (adesea ascuns). A face check out si a trimite commit-uri se rezolva prin operații obișnuite cu fișiere, precum copieri și redenumiri.

Sisteme centralizate pentru controlul versiunii

Sistemele locale pentru controlul versiunii au oferit o soluție pentru gestiunea versiunii pentru aplicații dezvoltate de o singură persoană pe un anume sistem. Din păcate această soluție nu vine în ajutorul persoanelor ce trebuie să colaboreze cu alți dezvoltatori și să interacționeze cu alte sisteme. Pentru a rezolva și această problemă sistemele centralizate pentru controlul versiunii (CVCS – Centralized Version Control Systems) au fost dezvoltate.

Sistemele centralizate pentru controlul versiunii (CVCS – Centralized Version Control Systems) (precum CVS, Subversion, Perforce etc) au repository-ul pe un singur server, conectat la retea. Pentru a face check out, fisierele trebuiesc transferate din server în directorul de lucru.

Aceasta soluție oferă numeroase avantaje comparativ cu sistemele locale pentru controlul versiunii: toată lumea știe până la un anumit nivel ce fac restul colaboratorilor în cadrul proiectului; administratorii au la dispoziție o manieră eficientă de a gestiona accesul la resurse și este mult mai ușor de gestionat o bază de date centrală decât una locală pentru fiecare utilizator.

O problemă evidentă este legată de disponibilitatea nodului central: dacă acesta întâmpină orice fel de problemă o să împiedice orice acțiune din partea utilizatorilor până problema va fi rezolvată. În plus fiind singurul sistem care conține toate stările fișierelor din cadrul unui proiect, în cazul în care aceste date sunt pierdute recuperarea acestora este de foarte multe ori imposibilă.

Sisteme distribuite pentru controlul versiunii

Dezavantajele și riscurile create de un sistem centralizat pentru controlul versiunii a condus la apariția sistemelor distribuite pentru controlul versiunii (Distributed Version Control Systems – DVCS)(precum Git, Mercurial, Bazaar, Darcs etc), în cadrul acestora utilizatorii nu obțin doar ultima versiune a fișierelor, ci propria copie a repository-ului cu toate versiunile noi și vechi ale proiectului. Diferitele repository-uri se sincronizează între ele periodic pe rețea.

Un avantaj este că această abordare adaugă robustețe. În modelele locale și centralizate, un repository corupt sau un hard disk eșuat poate duce la pierderea tuturor datelor. În modelul distribuit, vă puteți recupera adesea din astfel de accidentări, deoarece mai mulți dezvoltatori își pot uni repository-le pentru a recupera întregul istoric al proiectului. Majoritatea echipelor care lucrează cu VCS distribuite vor desemna în continuare un repository ca origine centrală a copiilor lor, pur și simplu pentru că acest lucru simplifică comunicarea în echipă.

Avantajele sistemelor de versionare

Care sunt avantajele folosirii unui sistem de versionare? este salvat istoricul tuturor modificărilor, astfel că se poate reveni oricând la o versiune mai veche dacă se descoperă prin folosirea unui serviciu de hosting, codul sursă are mereu o copie de siguranță online cea mai recentă versiune a codului sursă este mereu disponibilă tuturor dezvoltatorilor, făcând astfel colaborarea și sincronizarea mult mai ușoară decât în cazul trimiterii de fișiere conținând cod sursă dezvoltatorilor interesați de proiect.

GitHub

GitHub este o platformă online, pe care dezvoltatorii o pot folosi pentru a stoca și versiona codul lor sursă. Git este un sistem de management și versionare a codului sursă care permite lucrul eficient la un proiect software. Astfel, Git este utilitarul folosit în terminal, iar GitHub este serverul și aplicația web pe care rulează acesta, locul în care păstrăm repository-ul remote.

Primii pasi în git

Instrucțiunile de instalare git se regăsesc la link-ul https://git-scm.com/downloads [https://git-scm.com/downloads].

1. Crearea unui cont pe https://github.com [https://github.com]
2. Crearea unui token de autentificare
    a. Autentificați-vă pe GitHub.
    b. Mergeți aici [https://github.com/settings/tokens] pentru a genera un token de autentificare pentru GitHub.
    c. Apăsați pe Generate new token și introduceți parola voastră de GitHub.
    d. La câmpul Note scrieți un nume pentru token, ca să știți pentru ce îl puteți folosi ulterior.
    e. La câmpul Expiration selectați No expiration.
    f. La câmpul Select scopes bifați repo Full control of private repositories.
    g. Veți vedea o căsuță verde cu un șir de caractere, salvați-l. Acesta va fi folosit în loc de parolă atunci când vi se va cere parola în linia de comandă.
3. Pregătirea inițială a mediului Git

```
student@ic:~$ git config --global user.name "Prenume Nume"
student@ic:~$ git config --global user.email "adresa_de_email@example.com"
student@ic:~$ git config --list
```

4. Crearea unui repository gol pe GitHub
    - Apăsăm pe săgeata din meniul din dreapta sus, la profil.
    - Apăsăm pe Your profile pentru a merge pe profilul nostru.
    - Apăsăm pe Repositories și apasam pe New.
    - Pentru acest tutorial vom crea un repository privat.
    - Numele repository-ului va fi refactoring-lab.
    - Apăsăm pe Create repository.
5. Crearea unui repository gol local

```
student@ic:~$ mkdir refactoring-lab
student@ic:~$ cd refactoring-lab
student@ic:~$ git init
```

6. Conectarea celor două repository-uri

```
student@ic:~/refactoring-lab$ git remote add origin https://github.com/{username}/refactoring-lab.git
```

7. Primul fișier adaugat

```
student@ic:~/refactoring-lab$ echo "# Refactoring Lab" >> README.md
student@ic:~/refactoring-lab$ git status
student@ic:~/refactoring-lab$ git add README.md
student@ic:~/refactoring-lab$ git status
student@ic:~/refactoring-lab$ git commit -m "Add README file"
student@ic:~/refactoring-lab$ git status
student@ic:~/refactoring-lab$ git branch -M main
student@ic:~/refactoring-lab$ git push -u origin main
```

Refactoring

Clean Coding

Clean Coding este un set de principii și practici folosite de către programatori pentru a scrie un cod lizibil, de încredere și ușor de intretinut. Prin urmarirea acestor principii, codul va fi mai ușor de citit și îmbunătățit și de către alți dezvoltatori, nu doar de autor. Un cod bun este:

- Lizibil
- De incredere
    - A fost testat?
        - A trecut testele?
        - Cat de bine a fost testat?
    - Are code smell? Code smell este o secțiune din cod care poate avea un comportament neașteptat sau chiar periculos.
- Usor de intretinut
    - Este codul atat de strans cuplat încât o schimbare într-o parte afectează alte părți?
    - Exista cod duplicat ce necesita ca o schimbare sa fie aplicata simultan in mai multe locuri?

Principii

Un cod bun are în spate următoarele principii:

- Trebuie sa treaca toate testele
- Codul trebuie sa fie expresiv
- Keep it small, keep it simple

Un principiu de design important care reflectă toate aceste principii este SOLID din OOP, care este enunțat în “Design Principles and Design Patterns” de Robert C. Martin:

1. Single Responsibility - “There should never be more than one reason for a class to change.”
2. Open-Closed Principle - “Software entities … should be open for extension, but closed for modification.”
3. Liskov Substitution - “Functions that use pointers or references to base classes must be able to use objects of derived classes without knowing it.”
4. Interface Segregation - “Many client-specific interfaces are better than one general-purpose interface.”
5. Dependency Inversion - “Depend upon abstractions, [not] concretions.”

Refactor

Refactoring este procesul de a face schimbări în cod care-l fac mai expresiv dar nu-i schimbă comportamentul. Exemple de refactoring:

- redenumirea entităților
- inlocuirea literal constants prin named constant variables
- formatarea codului
- încapsularea câmpurilor publice ale claselor
- divizarea codului în funcții

Exemple:

- ItemMeaningful names

```
      int nCstCnt; // number of customers -> int numberOfCustomers;
```

Numele de variabile trebuie să exprime intenția noastră, să fie pronuntabil, să aibă distingeri clare. Te poți raporta la următoarea regulă: clasele, variabilele și constantele să fie subiecte, iar metodele și funcțiile să fie verbe.

Care este diferența dintre o clasă cu numele ‘Book’ și ‘BookData’? Ambele conțin informații despre obiectul Book.

- Limit your Scope

Declarați variabilele într-un scop cât mai limitat posibil: Un exemplu de așa nu:

```
String firstName;
String lastName;
int pos = fullName.find(',');
if (pos >= 0) {
        lastName = fullName.substr(0, pos);
        firstName = fullName.substr(pos+1);
        println ("Hello " + firstName + ' ' + lastName);
} else {
        println ("Hello " + fullName);
}
```

Scope-ul variabilelor firstName și lastName continuă existe pană la sfârșitul blocului de cod }

O soluție posibilă ar putea sa fie:

```
int pos = fullName.find(',');
if (pos >= 0) {
        String lastName = fullName.substr(0, pos);
        String firstName = fullName.substr(pos+1);
                println ("Hello " + firstName + ' ' + lastName);
        } else {
                println ("Hello " + fullName);
        }
```

- Functions should do One Thing

Nu este recomandată următoarea practică:

```
public void addBookToCollection (Book book)
{
        List<Author> authors = book.getAuthors();
        Title title = book.getTitle();
        List<SubjectKeyword> subjects = new List<>();
                try {
                        subjects = book.getSubjects();
                } catch (SubjectsUnassigned ex) {
                        requestReviewOfSubjects(book);
                }
                try {
                        DecimalClassification dewey = oclc.getClassificationCode(book.getISBN());
                        ShelfLocation intendedLocation = getShelfFor(dewey);
```

```
                        book.setLocation (intendedLocation);
                } catch (OCLCServiceUnavailable ex) {
                        log.error("Error contacting OCLC about " + book.toString(), ex);
                }
                try {
                        for (Author author: authors) {
                                authorIndex.add(author, book);
                }
                        titleIndex.add (title, book);
                        for (SubjectKeyword subject: subjects) {
                                subjectIndex.add(subject, book);
                }
                         } catch (CatalogingError ex) {
                                  log.error ("Error cataloging " + book.toString(), ex);
                                  }
}
```

O soluție posibilă ar putea fi:

```
        public void addBookToCollection (Book book)
        {
          addBookToCatalogs(book);
          moveBookToIntendedLocation(book);
        }

        private void addBookToCatalogs(book) {
                addBookToAuthorCatalog(book);
                addBookToTitleCatalog(book);
                addBookToSubjectCatalog(book);
        }

        private void addBookToAuthorCatalog(Book book) {
                List<Author> authors = book.getAuthors();
                try {
                        for (Author author: authors) {
                                authorIndex.add(author, book);
                        }
                 } catch (CatalogingError ex) {
                          log.error ("Error cataloging " + book.toString(), ex);
                 }
        }

        private void addBookToTitleCatalog(Book book) {
                Title title = book.getTitle();
                try {
                        titleIndex.add (title, book);
                 } catch (CatalogingError ex) {
                          log.error ("Error cataloging " + book.toString(), ex);
                 }
        }

        private void addBookToSubjectCatalog (Book book) {
                try {
                        List<SubjectKeyword> subjects = book.getSubjects();
                        for (SubjectKeyword subject: subjects) {
                                subjectIndex.add(subject, book);
                        }
                 } catch (CatalogingError ex) {
                          log.error ("Error cataloging " + book.toString(), ex);
                 }
                 } catch (SubjectsUnassigned ex) {
                        requestReviewOfSubjects(book);
                 }
                }


                private void moveBookToIntendedLocation (Book book) {
                        try {
                                ShelfLocation intendedLocation = getIntendedLocation(book);
                                book.setLocation (intendedLocation);
                        } catch (OCLCServiceUnavailable ex) {
                                log.error("Error contacting OCLC about " + book.toString(), ex);
                        }
                }

                private ShelfLocation getIntendedLocation (Book book)
                          throws OCLCServiceUnavailable
```

```
{
        DecimalClassification dewey = oclc.getClassificationCode(book.getISBN());
        return getShelfFor(dewey);
}
```

- Evitarea duplicarii

Nu este recomandată următoarea practică

```
private void addBookToCatalogs(book) {
        attemptToAddBookToAuthorCatalog(book);
        attemptToAddBookToTitleCatalog(book);
        attemptToAddBookToSubjectCatalog(book);
}

private void attemptToAddBookToAuthorCatalog(Book book) {
        try {
                addBookToAuthorCatalog(book);
         } catch (CatalogingError ex) {
                  log.error ("Error cataloging " + book.toString(), ex);
         }
}

private void attemptToAddBookToTitleCatalog(Book book) {
        try {
                addBookToTitleCatalog (book);
         } catch (CatalogingError ex) {
                  log.error ("Error cataloging " + book.toString(), ex);
         }
}

private void attemptToAddBookToSubjectCatalog(Book book) {
        try {
                addBookToSubjectCatalog (book);
         } catch (CatalogingError ex) {
                  log.error ("Error cataloging " + book.toString(), ex);
         }
}
```

O soluție posibilă ar putea fi:

```
private void attemptToAddBookToCatalogs(book) {
                try {
                addBookToCatalogs(book);
         } catch (CatalogingError ex) {
                log.error ("Error cataloging " + book.toString(), ex);
         }
}

private void addBookToCatalogs(book) {
        addBookToAuthorCatalog(book);
        addBookToTitleCatalog(book);
        addBookToSubjectCatalog(book);
}
```

- Legea lui Demeter

The law of Demeter: a method f of a class C should only call the methods of:

- C
- an object created by f
- an object passed as an argument to f
- an object held in a (data structure of a) data member of C
- Error Handling
- e mai bine sa folosesti try/catch si exceptii decat sa returnezi NULL sau codui de erori specifice.

Exercitii

Descarcati arhiva de aicigapsflat.zip.

1. Deschideți proiectul și încercați să rulați cu argumentul testData/test000.dat
2. Refactoring: Project Layout
    - Fiecare proiect Java trebuie să conțină un director src în care sunt prezente fișierele sursă .java. Creați directorul src/main/java și adăugați sursele în acesta.
    - Creați directorul src/test/data în care adaugați fisierele de test
    - Marcati directorul src ca unicul director sursa din File → Project Structure… → Modules
3. Refactoring : Improving the Source Code
    - Deschideți Highway.java și cautați unde este apelată funcția subtract
    - Pe funcție dați click dreapta → Go To → Implementation(s) (scurtătura CTRL + click stanga) pentru a ajunge la implementarea funcției
    - Redenumiți funcția din “substract” în “remove”.
    - Conform principiilor de Clean Coding, noua functie “remove” are prea multe conditii. Selectați al doilea if din while și dați click dreapta→Refactor→Extract Method și numiți funcția replaceIfOverlapping.
    - Același lucru îl vom face cu body-ul primului if din noua funcție replaceIfOverlapping și metoda o vom numi replaceOverlappingInterval
    - Cum am făcut mai sus vom face și pentru if-urile din replaceOverlappingInterval, și metoda o vom numi addIfNonEmpty
4. Verificați dacă rulează.
5. Dati push modificărilor pe Github prin CLI.

Posibile probleme:

- Nu este configurat SDK (File→Project Structure→Project și alegeți SDK-ul)
- Nu este configurat source folder-ul (File→Project Structure→Modules alegeți directorul și marcați-l ca Sources)

Resurse utile

- https://gitimmersion.com/ [https://gitimmersion.com/]
- https://git-scm.com/docs/ [https://git-scm.com/docs/]

Referințe

- https://www.atlassian.com/git/tutorials/what-is-version-control [https://www.atlassian.com/git/tutorials/what-is-version-control]
- https://teaching.alexcoman.com/resurse/tutorial/git/sisteme-pentru-controlul-versiunii/ [https://teaching.alexcoman.com/resurse/tutorial/git/sisteme-pentru-controlul-versiunii/]
- https://www.cs.odu.edu/~zeil/cs350/latest/Public/versionControl/index.html [https://www.cs.odu.edu/~zeil/cs350/latest/Public/versionControl/index.html]

icalc/laboratoare/laborator-05.txt · Last modified: 2022/04/18 22:50 by liviu.mitruta

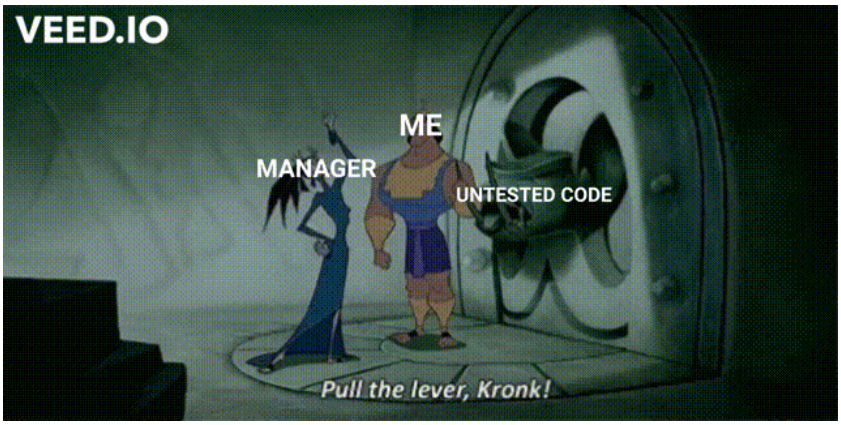
Laborator 06 - Unit Testing

Set up

Best to have: **InteliJ**

- Instalare: https://ocw.cs.pub.ro/courses/poo-ca-cd/resurse-utile/instalare-intellij-idea [https://ocw.cs.pub.ro/courses/poo-ca-cd/resurse-utile/instalare-intellij-idea]
- Activare: https://ocw.cs.pub.ro/courses/poo-ca-cd/resurse-utile/activare-intellij-idea [https://ocw.cs.pub.ro/courses/poo-ca-cd/resurse-utile/activare-intellij-idea]

Introducere



De ce testam?

Testam pentru a arata ca **programul functioneaza**. Acest aspect este logic, dar are cateva **probleme**:

- Are tendinta de a se transforma intr-o “profetie”. Cel mai simplu mod de a-ti trece testele si de a te convinge ca programul tau functioneaza este sa alegi un set minim de teste si sa nu incerci foarte mult.
- Incercarea de a arata ca un program functioneaza prin testare este, intr-un fel, inutila. Testarea programelor poate dezvalui in mod eficient prezenta erorilor, dar niciodata sa nu arate absenta acestora.

Testarea este, in esenta, un **proces de esantionare**.

Ganditi-va la un sac plin cu pietre, pietre albe indicand o intrare pentru care programul reuseste, pietre negre indicand o intrare pentru care programul esueaza.

Acum sa presupunem ca ati scos pietre din geanta si ca ati vazut doar pietre albe.

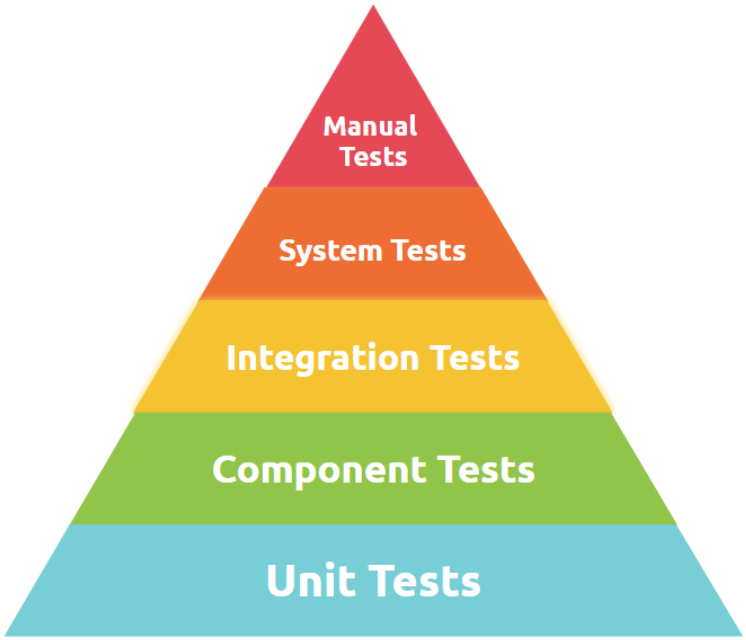
Cate trebuie sa extragi inainte de a putea concluziona cu siguranta ca nu exista pietre negre in geanta?

De fapt, singura modalitate de a fi sigur de asta este sa scoti fiecare piatra din sac. Dar daca sacul are atat de multe pietre incat nu avem timp si rabdare sa scoatem mai mult decat un mic procent… Ei bine, pur si simplu nu putem fi siguri ca nu sunt mai multe pietre negre aflate in sac.

Deci **„De ce testam?”**. Testam pentru **a castiga incredere in codul nostru**. Daca alegem o multime de teste si le alegem intr-un mod inteligent, putem spune in mod legitim ca am invatat ceva. Poate ca nu putem spune ca nu au mai ramas bug-uri, dar putem spune cel putin ca programul functioneaza de cele mai multe ori.

Moduri de testare

Piramida testelor



Teste unitare

Testele unitare sunt teste automatizate care asigura ca o aplicatie indeplinește **cerintele de design** si aplicatia ca **se comporta conform specificatiilor**.
In ceea ce priveste OOP, “unitatea” care urmeaza sa fie supusa testarii este adesea:

- **interfata**
- **clasa**
- **metoda**

Junit 5

- JUnit - un framework de testare unitară pentru Java.
- Important in Test-driven development(TDD).

TDD (Test Driven Development) este un sproces de dezvolatre software care se bazeaza pe repetarea a unui ciclu foarte scurt de dezvoltare: Cerintele sunt transformate in cazuri de testare specifice → software-ul este imbunatatit doar pentru a trece DOAR testele noi.
Beneficii TDD:

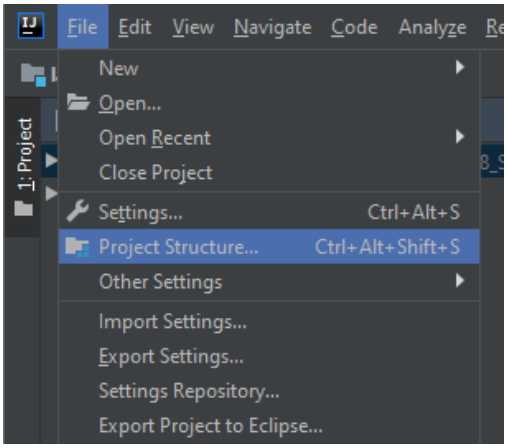
- “Driven” de catre obiective clare
- Cod mai sigur
- Izoleaza codul incorect
- Se introduc cu usurinta functionalitati noi
- Se documenteaza aplicatia.

- JUnit este conectat ca JAR la momentul compilarii.
- JUnit este cea mai frecvent inclusa biblioteca externa în proiecte Java.

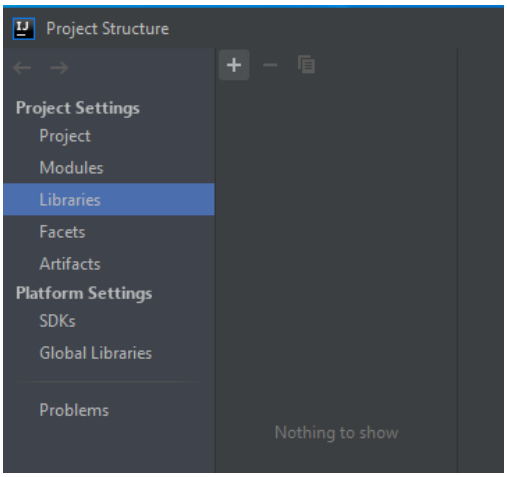
JUnit 5 este ultima versiune a bibliotecii JUnit. JUnit 5 este compatibil cu **Java 8 sau orice versiune dupa aceasta**.

Adaugare JUnit in IntelliJ

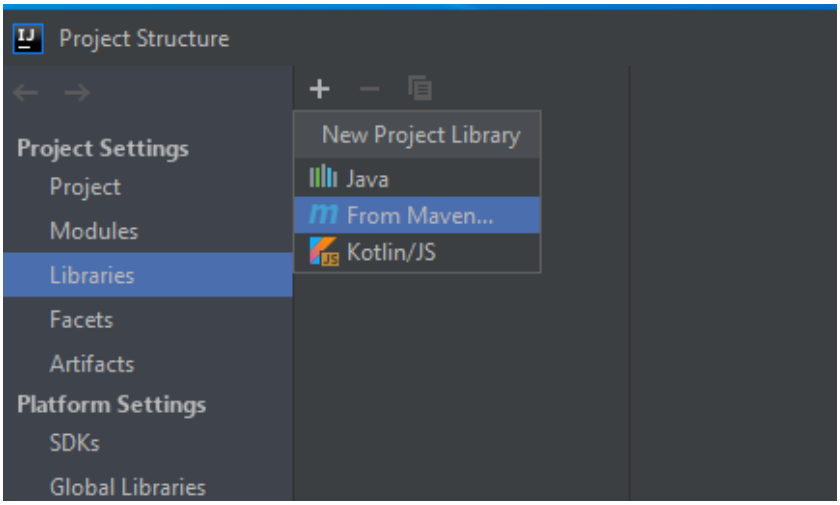
1. IntelliJ: **File → Project Structure**



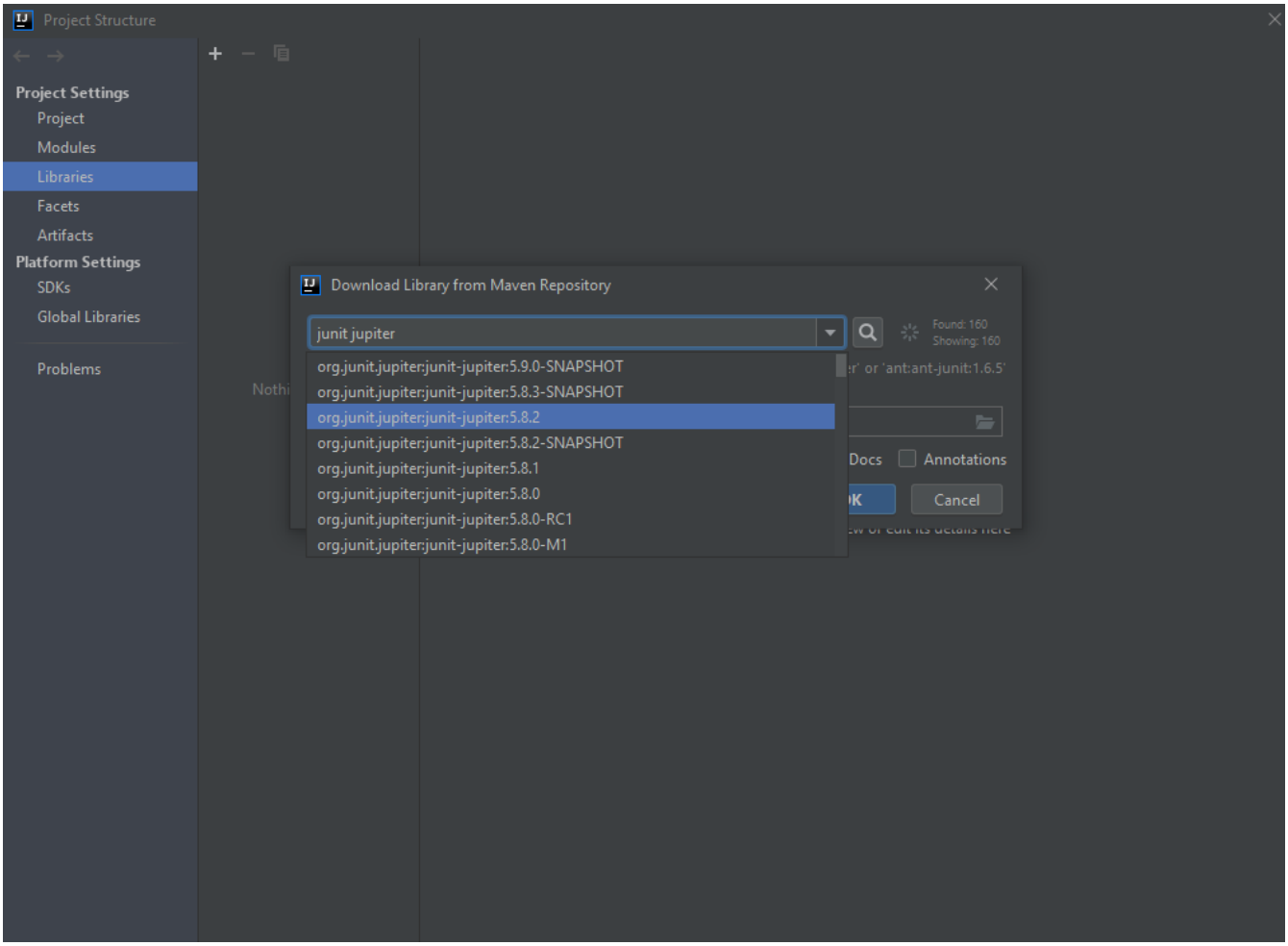
2. Project Structure: **Libraries**



3. Libraries: **+ → From Maven…**



4. **Tastati** in bara de search '*junit jupiter*', iar apoi apasati **Enter**. **Selectati**: *org.junit.jupiter:junit-jupiter:5.8.2*



5. Apasati **OK**, iar apoi asteptati descarcarea bibliotecii.

6. In final din fereastra de **Project Structure**, apasati **Apply**, ai apoi **OK**.

Creare teste

Crearea unui test se realizeaza prin ***adnotarea*** unei metode cu ***@Test***.

```
@Test
public void theySeeMeRollinTheyHatin() {
    /* code to test */
}
```

Sa presupunem ca avem o clasa care contine informatii despre un student:

```
public class Student {
    private String name;
    private Double gradeIC;
    private boolean lovesIC;
    private LocalDate birthday;

    public Student(String name, Double gradeIC, boolean lovesIC, LocalDate birthday) {
        this.name = name;
        this.gradeIC = gradeIC;
        this.lovesIC = lovesIC;
        this.birthday = birthday;
    }

    public void sprinkleSomeMagicNerdDust() {
        this.lovesIC = true;
        this.gradeIC = 10.0;
    }

    ...
    /* boilerplate code aka getters and setters */
    ...
}
```

Si dorim sa cream un test pentru metoda, constructor, getters si setters.

Pentru a realiza acest lucru, avem nevoie sa folosim cateva metode din biblioteca de JUnit. Cele mai **relevante metode** pe care le vom folosi pentru a testa, sunt:

assertEquals(Object **expected**, Object **actual**);

assertNotEquals(Object **unexpected**, Object **actual**);

assertNotNull(Object **actual**);

Exista o colectie generoasa de metode prin care se poate face assert: https://junit.org/junit5/docs/5.0.1/api/org/junit/jupiter/api/Assertions.html [https://junit.org/junit5/docs/5.0.1/api/org/junit/jupiter/api/Assertions.html]

Astfel, testul nostru o sa arate in modul urmator:

```
@Test
public void noLongerATestNoobie() {
    Student student = new Student("Chris Rock", 7.0, false, LocalDate.of(1965, 2, 7));
    assertNotNull(student); // True
    assertEquals(student.getGradeIC(), 7.0); // True
    assertEquals(student.getLovesIC(), false); // True

    student.sprinkleSomeMagicNerdDust();

    assertNotEquals(student.getGradeIC(), 7.0); // True => no longer 7. the previous call changed the grade.
    assertEquals(student.getGradeIC(), 10.0); // True
    assertEquals(student.getLovesIC(), true); // True
}
```

In cazul in care **toate assert-urile** o sa fie **True**, testul a **trecut**. In cazul in care oricare dintre assert-uri intoarce False, atunci testul o sa pice.

Alte metode uzuale de assert sunt:

- **assertThat**

```
/* e.g. */
assertThat(student, instanceOf(Student.class));
```

- **assertAll**

```
/* e.g. */
assertAll("last name", () -> assertTrue(student.getName().startsWith("C")),
                       () -> assertTrue(student.getName().endsWith("k")),
                       () -> assertEquals(7.0, student.getGradeIC()));
```

- **assertThrows**

```
/* e.g. */
NumberFormatException thrown = assertThrows(NumberFormatException.class, () -> { Integer.parseInt("One"); } ,
                                            "NumberFormatException was expected");

Assertions.assertEquals("For input string: \"One\"", thrown.getMessage());
```

In cazul in care nu se face import automat pentru @Test si assert, adaugati:

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
```

In ceea ce priveste **numirea metodelor de test**, cea mai utilizata conventie este: **MethodName_StateUnderTest_ExpectedBehavior**. (e.g. isAdult_AgeLessThan18_False). Mai multe conventii gasiti la linkul: https://dzone.com/articles/7-popular-unit-test-naming [https://dzone.com/articles/7-popular-unit-test-naming]

In cazul in care doriti sa rulati testele din command line:
java -jar junit-platform-console-standalone-<*version*>.jar <*Options*>

Before si After

Sa spunem ca ne dorim sa existe un anumit set up pentru testele noastre, sau ca vrem avem un anumit comportament dupa finalizarea testului.
Am putea adauga aceste functionalitati in corpului metodei de test. Dar daca avem mai multe teste pentru care am vrea sa se intample acelasi lucru inainte sau dupa test?
Am ajunge in situatia in care am avea mult cod duplicat, iar asta ar incalca un pricipiu important de dezvoltare - **DRY** (Don't repeat yourself).

In acest caz, vom introduce cateva adnotari noi:
@BeforeAll si **@AfterAll**, respectiv **@BeforeEach** si **@AfterEach**.

```
    @BeforeEach
    void setUp() {
        System.out.println("Test started");

        student = new Student("BoJack", 4.0, false, LocalDate.of(1964, 1, 2));
    }

    @Test
    void isPassingIC_GradeLessThan5_False() {
        assertNotNull(student);
        asssertFalse("Bro, that Gandalf meme is getting old...", student.isPassing()); // grade is 4 < 5 => student not passing
    }
```

```
    @Test
    void isPassingIC_GradeGraterThan5_True() {
        assertNotNull(student);
        student.study(); // method body: student.grade = 10;
        asssertTrue("Ops, i did it again!", student.isPassing()); // grade is now 10
    }

    @AfterEach
    void tearDown() {
        student.throwAParty();
        System.out.println("So cool, this is!");
    }
```

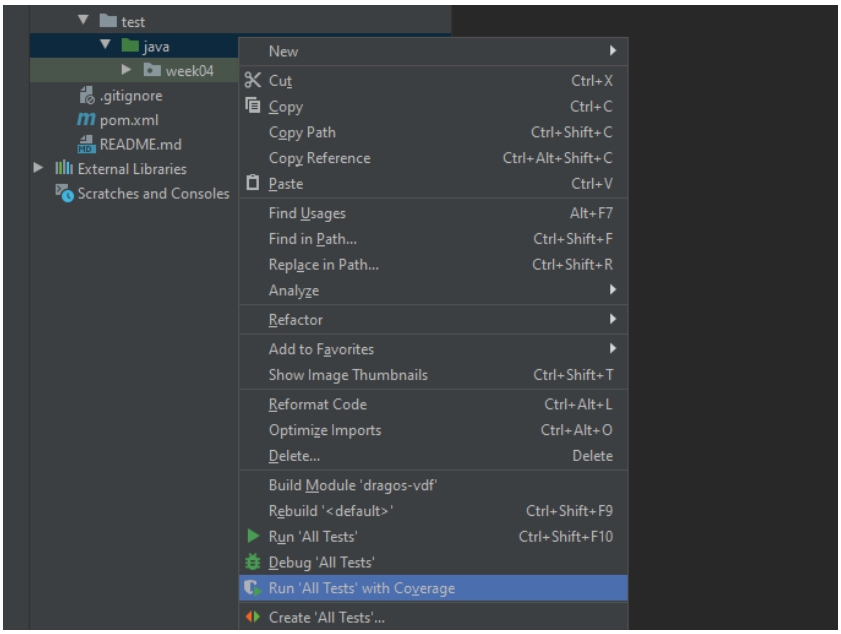
Coverage

Code coverage este o metrica care arata cat de mult din cod este testat. Este o metrica foarte utila care ajuta in evaluarea calitatii testelor.

Valorile comune care apar in rapoartele de acoperire includ:

- **Function coverage**: cate dintre funcțiile definite au fost apelate.
- **Statement coverage**: cate dintre instrucțiunile din program au fost executate.
- **Branches coverage**: cate *ramuri* ale structurilor de control (e.g if-else) au fost executate.
- **Condition coverage**: cate dintre subexpresiile booleene au fost testate pentru true si false.
- **Line coverage**: cate linii de cod sursa au fost testate.

Pentru a rula coverage pe un test in IntelliJ:
Selectati clasa de test/folder-ul de teste → **Click** dreapta → Click **'Run “Tests” With Coverage'**



Exercitii

1. **Descarcati** arhiva de laborator: laborator6.zip.
2. **Importati** arhiva ca proiect in IntelliJ.
3. **Parcurgeti** continutul din *src→main→java→org→catalogic*.
4. Dupa ce ati parcurs codul sursa, **intrati** in *src→test→java→org→catalogic*.
5. In acest package, veti avea **4 suite de teste**:
    - (2p) **Test1**: in aceasta clasa completati testele pentru a va asigura ca factory-ul creeaza instante corecte pentru obiectele voastre.
    - (3p) **Test2**: in aceasta clasa completati testele pentru a va asigura ca se arunca exceptiile potrivite atunci cand este cazul.
    - (4p) **Test3**: in aceasta clasa completati testele pentru a va asigura ca funtionalitatea de addGrade functioneaza corespunzator.
    - (1p) **Test4**: rulati coverage-ul, iar in aceasta clasa completati cu toate testele necesare pentru a avea un coverage de 100%.

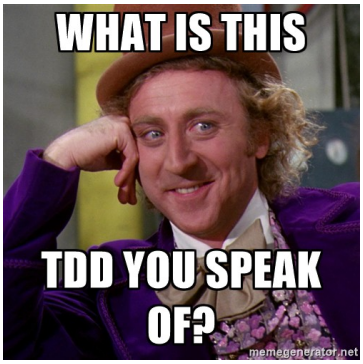
icalc/laboratoare/laborator-06.txt · Last modified: 2022/05/02 16:21 by dragos.sandulescu97

Laborator 07 - Test-driven development

Best to have: **InteliJ**

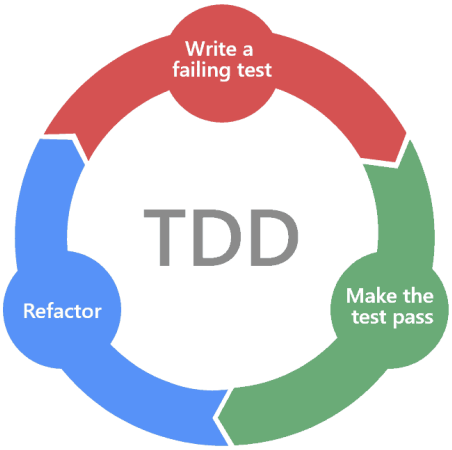
- Instalare: https://ocw.cs.pub.ro/courses/poo-ca-cd/resurse-utile/instalare-intellij-idea [https://ocw.cs.pub.ro/courses/poo-ca-cd/resurse-utile/instalare-intellij-idea]
- Activare: https://ocw.cs.pub.ro/courses/poo-ca-cd/resurse-utile/activare-intellij-idea [https://ocw.cs.pub.ro/courses/poo-ca-cd/resurse-utile/activare-intellij-idea]

Introducere



De cele mai multe ori, dupa ce un programator finalizeaza scrierea unui “program” dupa anumite specificatii, acesta este trimis catre echipa de testare in vedere gasirii bug-urilor. Daca sunt gasite probleme, “programul” se intoarce in faza de dezvoltare pentru a fi remediate. Acest schimb intre programator si echipa de testare se poate face de un numar suficient de mare de ori pana cand toate problemele sa fie rezolvate. Pentru a evita situatia enumerata mai sus se poate folosi tehnica TDD prin care programatorul scrie teste inainte de a implementa o anumita functionalitate.

Ce inseama TDD?



TDD (Test Driven Development) este o abordare a dezvoltarii de software prin care testele sunt scrise inainte ca softul sa fie scris. Cu alte cuvinte, este o cale de a gândi prin prisma cerințelor sau a designului înainte de a ne apuca efectiv a scrie cod funcțional.Un alt punct de vedere este că TDD reprezintă o tehnică de programare al cărui scop este acela de a scrie cod curat care funcționează. Aceasta metoda contine 3 pasi:

1. Se scrie un test care sa pice
2. Se scrie minimul de cod necesar pentru a face testul sa treaca
3. Se curata codul scris si se refactorizeaza daca este nevoie

La o prima analiza a acestei metode ar parea ca este o pierdere de timp prin faptul ca trebuie scrise si teste si astfel dezvoltarea software-ului este incetinita. Dar aici vine marele avantaj. Dupa cum observam in pasii anteriori, scrierea unui test care esueaza este in stransa legatura cu dezvoltarea. Astfel, dupa ce scriem suficient cod pentru a face testul sa mearga (sau pana cand credem ca este suficient sa treaca testul) trebuie sa testam din nou. Avand momentele de timp, cand codul functioneaza fara erori, foarte bine stabilite in timp este mult mai usor sa revii intr-o stare in care codul este functional si sa rescrii functionalitatea ce esueaza. Astfel, se castiga timpul pierdut cu debugging-ul si cu gasirea functionalitatilor care esueaza. Mereu testul ce esueaza este ultimul si prin urmare problema este la ultima functionalitate adaugata.

Concluzii

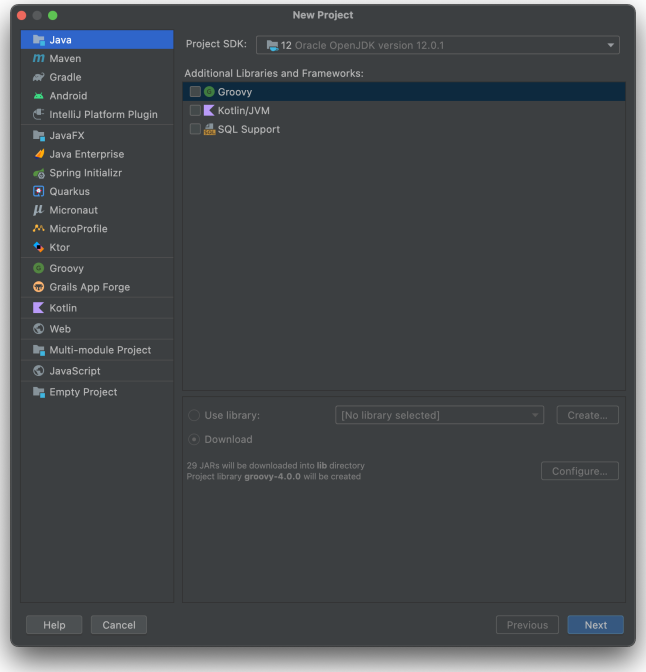
Test-driven development nu inlocuieste metoda traditionala de testare, dar defineste o modalitate de dezvoltare a codului in vederea limitarii greselilor. Mai mult, aceasta metoda aduce avantaje de timp pentru identificarea problemelor la introducerea functionalitatilor noi, permite dezvoltarea unui cod mai mic si mai usor de inteles si faciliteaza schimbul de cunostinte din cadrul unei echipe.

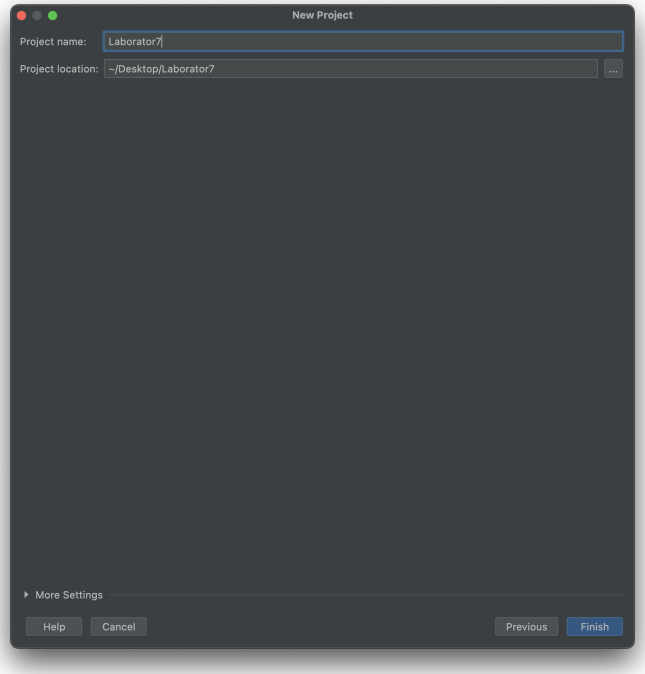
Cum pregatim proiectul?

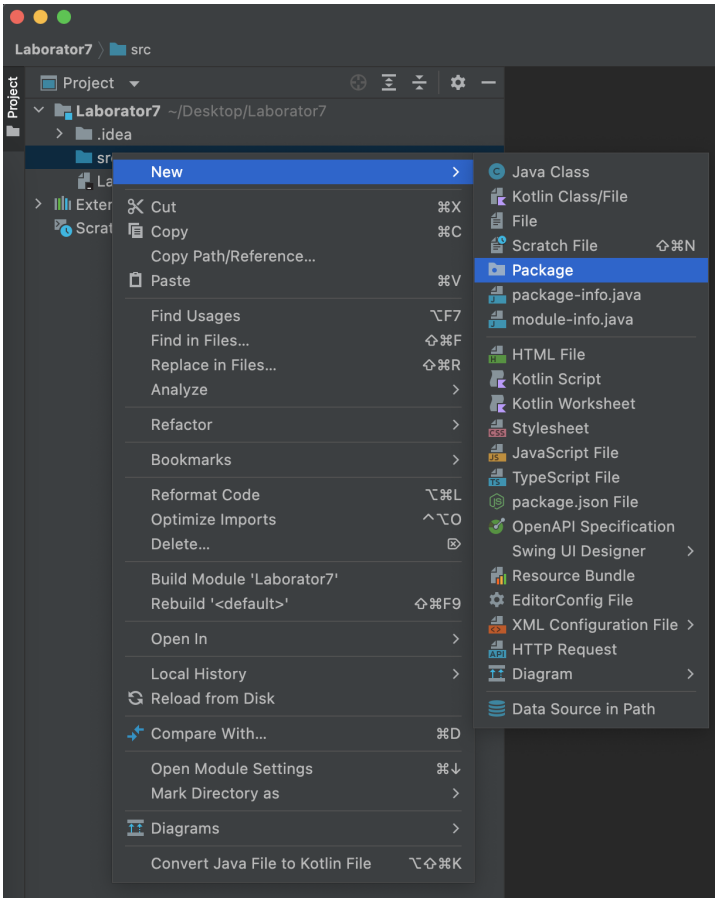
In cadrul acestui laborator vom face de la 0 un proiect si vom urma toti pasii necesari pentru a dezvolta software folosind TDD.

- Folosind interfata IntelliJ vom face un nou proiect denumit Laborator7

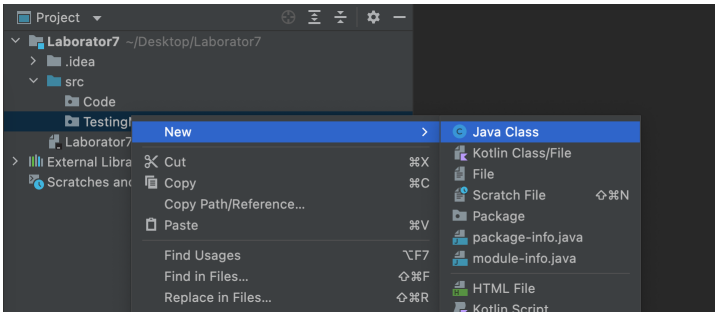
Pentru a separa partea de cod de partea de testare vom face 2 package-uri. In package-ul denumit **Code** vom adauga clasele ce vor fi folosite de software, iar in package-ul **TestingModule** vom adauga partea de testare.





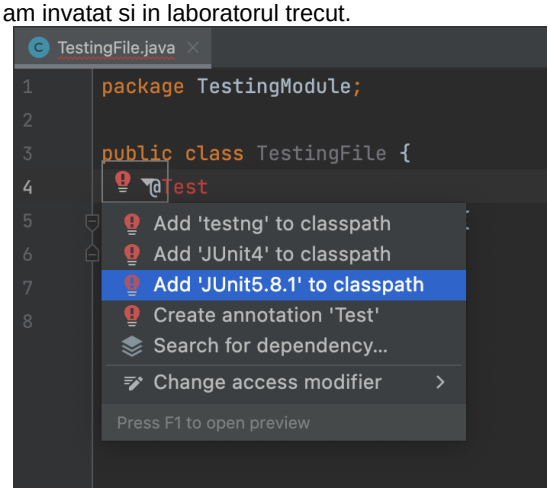


- Pentru a scrie prima functie de testare vom merge in package-ul **TestingModule** si vom face un nou fisier de tipul **Java Class**



- In cadrul acestui fisier facem o clasa in interiorul careia ne vom scrie testele. Crearea unui test se realizeaza prin adnotarea unei metode cu **@Test**.

In cazul in care IntelliJ nu importa automat modulele necesare putem sa le adaugam manual asa cum este prezentat in poza. Desigur, putem adauga **JUnit** in **IntelliJ** asa cum am invatat si in laboratorul trecut.



Specificatiile proiectului

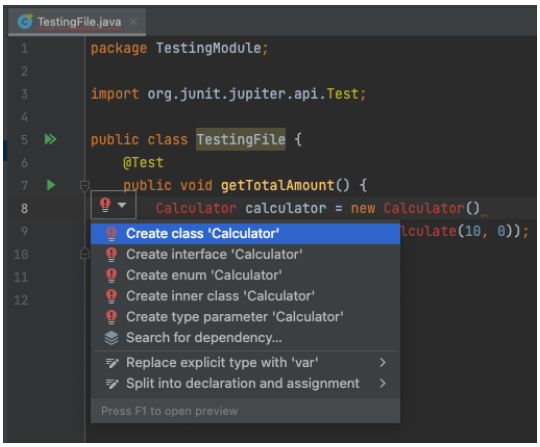
In cadrul laboratorului de astazi vom face un program denumit sugestiv **SplitBill**. Acesta va lua anumite sume de bani, persoane, procentul de tips si va returna o suma de bani ce trebuie data de fiecare persoana pentru achitarea notei.

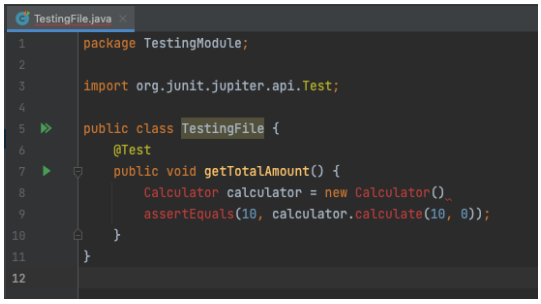
Cerinta 1

Etapa 1: writing a failing test

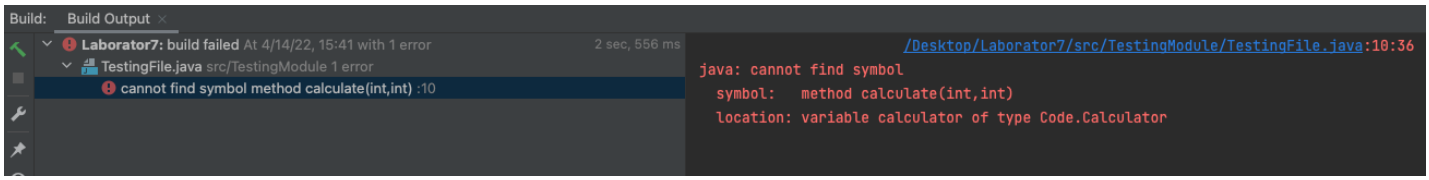
- Prima cerinta a proiectului este scrierea unui test **getTotalAmountWithZeroTips()**. In cadrul acestei functii vom lua o instanta a calculatorului si vom returna rezultatul unei metode **getTotalAmount(double amount, double tips)**. Aceasta va adauga procentul de tips la suma si va returna suma totala ce trebuie platita, suma ce contine si tips-ul.

IntelliJ ne ajuta si ne face highlight pe ce nu exista. Astfel ne putem folosi de editor pentru a face clasa Calculator si a adauga in ea lucrurile necesare.





Primul din cei 3 pasi TDD a fost realizat: testul a fost scris. Dupa ce rulam acest test vom vedea faptul ca testul nu va trece intrucat noi nu avem implementata aceasta metoda in cadrul clasei. Urmatorul pas este scrierea celor mai simple linii pentru a face testul sa treaca.

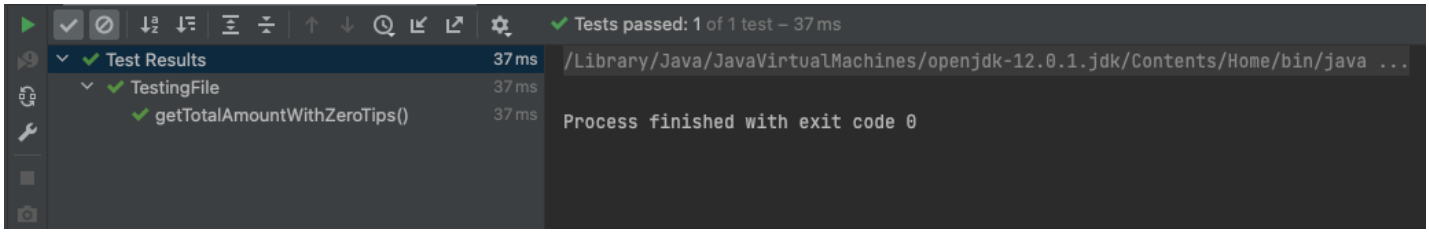


Etapa 2: make the test pass

Avand in vedere ca testul nostru este conceput pentru cazul in care tips-ul este 0 ceea ce avem de facut este sa returnam suma initiala:

```
public class Calculator {
    public double calculate(double amount, double tips) {
        return amount;
    }
}
```

Daca rulam testul inca odata vom vedea ca acesta va trece:



Etapa 3: refactoring

Pentru testul curent, dat fiind faptul ca este foarte simplu, vom omite etapa refactoring considerand ca nu avem ce modifica.

Cerinta 2

Etapa 1: writing a failing test

- A doua cerinta este scrierea unui test **getTotalAmountWithTips()**. De data aceasta avem ca date de intrare **amount 78** si **tips 10** si ne asteptam ca functia sa returneze 85.8.

```
@Test
public void getTotalAmountWithTips() {
```

```
    Calculator calculator = new Calculator();
    assertEquals(85.8, calculator.calculate(78, 10));
}
```

Etapa 2: make the test pass

```
public class Calculator {
    public double calculate(double amount, double tips) {
        if (tips == 0.0) {
            return amount;
        }
        else {
            var tipsAmount = amount / 100 * tips;
            return amount + tipsAmount;
        }
    }
}
```

Etapa 3: refactoring

De data aceasta putem face refactoring eliminand din codul redundant:

```
public class Calculator {
    public double calculate(double amount, double tips) {
        var tipsAmount = amount / 100 * tips;
        return amount + tipsAmount;
    }
}
```

Tips and Tricks

JUnit ne permite sa dam argumente functiilor de testare. Folosind adnotarea **@ValueSource** putem adauga un array de elemente pe care sa le folosim ca argumente.

```
    @ParameterizedTest
    @ValueSource(doubles = {0, 10, 100, 78, 20})
    public void getTotalAmountWithTips2(Double amount) {
        Calculator calculator = new Calculator();
        System.out.println("Amount " + amount + " with tips 10 = " + calculator.calculate(amount, 10));
    }
```

Folosind adnotarea **@CsvSource** putem da mai multe argumente functiilor de testare.

```
@ParameterizedTest
@CsvSource({
            "0, 0",
            "10, 11",
            "100, 110",
            "78, 85.8",
            "20, 22",
    })
    public void getTotalAmountWithTips3(Double amount, Double expected) {
        Calculator calculator = new Calculator();
        assertEquals(expected, calculator.calculate(amount, 10));
}
```

Este important de mentionat faptul ca atunci cand se folosesc adnotarile mentionate mai sus trebuie sa folosim **@ParameterizedTest** in loc de **@Test**.

Exercitii

Descarcati arhiva de aici: laborator7.zip.

Partea 1:

1. Sa se scrie un failing test **testWithMultipleAmounts** ce va lua un array de amount-uri (double) si va returna suma ce contine si tipsul.
2. Sa se scrie codul (si sa se refactorizeze daca este nevoie) pentru a face testul de la pasul anterior sa treaca.
3. Sa se scrie un test ce verifica faptul ca daca exista sume negative sau procentul de tips este negativ atunci se arunca o eroare.
4. Sa se scrie codul (si sa se refactorizeze daca este nevoie) ce face testul mentionat mai sus sa treaca.
5. Sa se scrie un test pentru valorile de intrare **amount = 100**, **tips = 10** si **personNumber = 2** rezultatul fiind **expected = 55**.
6. Sa se scrie codul necesar pentru a face testul mentionat mai sus sa treaca. (trebuie adaugat suport pentru un parametru in plus si anume numarul de persoane intre care trebuie impartita nota).

7. Sa se scrie un test folosind adnotarea **@CsvSource** prin care sa se testeze 5 cazuri de intrare diferite modificand toti parametrii de intrare (lista de sume, tipsul si numarul de persoane).

Partea 2:

1. Creati un proiect nou sau folositi scheletul din cadrul primei parti.
2. Sa se scrie un test ce verifica daca suma numerelor dintr-un string gol este 0.
3. Sa se implementeze functia ce face testul sa treaca (si sa se refactorizeze daca este nevoie).
4. Sa se scrie o functie de test ce verifica faptul ca suma cu stringul de intrare **“1 2 3 10 4 7”** este **27**. (numerele sunt separate prin exact un spatiu.)
5. Sa se implementeze codul ce face testul de mai sus sa treaca (si sa se refactorizeze daca este nevoie).
6. Sa se scrie o functie ce testeaza folosirea ”,” ca si delimitator in loc de spatiu.
7. Sa se scrie codul ce face testul de mai sus sa treaca (si sa se refactorizeze daca este nevoie).
8. Sa se scrie o functie de test ce permite folosirea literelor (mari si mici) si semnelor de punctuatie (,.?!:;) ca si delimitatori. Pot exista oricat de multi delimitatori intre numere.
9. Sa se implementeze codul ce face testul de mai sus sa treaca (si sa se refactorizeze daca este nevoie).

Refactorizare: folositi regex

Resurse utile

- https://www.jetbrains.com/help/idea/tdd-with-intellij-idea.html [https://www.jetbrains.com/help/idea/tdd-with-intellij-idea.html]
- https://www.baeldung.com/parameterized-tests-junit-5 [https://www.baeldung.com/parameterized-tests-junit-5]
- https://justinbois.github.io/bootcamp/2020/lessons/l00_configuring_your_computer.html [https://justinbois.github.io/bootcamp/2020/lessons/l00_configuring_your_computer.html]

icalc/laboratoare/laborator-07.txt · Last modified: 2022/05/11 20:52 by theodor.ungureanu

Laborator 08 - CI & Documentation

Obiectiv

În prima secțiune vom discuta despre comentariile din codul sursa si cum se documenteaza un API. In a doua parte vom vorbi de Continuous Integration, in ce consta si moduri de a analiza codul.

Documentatia

Comentariile

Comentariile sunt printre cele mai folosite si abuzate moduri de a documenta codul. Acestea pot fi folosite pentru a ascunde diverse imperfectiuni ale codul sau proiectului ca nume neinspirate ale variabilelor sau functiilor, design slab al codului sau lipsa uneltelor de proces (sistem de versionare, issue tracking etc.). Comentariile pot fi utile pentru a explica de ce este facut un lucru in cod, pentru a documenta un design bazat pe pseudo-cod sau pentru a referentia lucruri conexe.

Tipuri de comentarii

- Repetare a codului
  - Inutile
- Explicarea codului
  - Utile in cazul in care codul nu este clar, dar in acest caz prioritatea ar fi clarificarea codului
- Markers
  - Note menite sa fie scoase din codul final
  - Utile daca sunt standardizate ex. `// TODO`
- Rezumat al codului
  - Aplicate pentru sectiuni mai mari de cod
  - Utile pentru a intelege mai repede si usor codul
- Descriere a unei potentiale probleme
  - Asemanator cu rezumatul, doar ca se descrie o potentiala problema
- Alte informatii care nu pot fi exprimate in cod
  - Numele autorilor, copyright, data modificarii etc.

Documentarea unui API

Uneltele de documentare a API-urilor au devenit extrem de comune in prezent. Acestea:

- Reflecta accentul modern pus pe interfetele reutilizabile
- Combina informatii de la
  - Un parser limitat de limbaj (pentru a extrage informatii despre structura modulului/functiei si parametrii asociati
  - Sectiuni de comentarii special formatate din codul sursa
- Incurajeaza actualizarea comentariilor pe masura ce codul este modificat

JavaDoc

Una dintre cele mai cunoscute unelte de documentare a API-urilor pentru ca este inclusa in distributia standard de Java. Comentariile specifice JavaDoc:

- Sunt delimitate de `/** ... */`, implicit procesate ca si comentarii de compilator
- Preced sectiunea de cod pe care o comenteaza
- Pe langa orice text, acestea pot contine marcaje speciale cum ar fi `@author <authorName>`, `@param <name> <description>`, `@return <description>` etc.

Pentru a genera documentatia folosind JavaDoc se poate folosi:

- Linia de comanda `javadoc -d destinationDir -sourcepath sourceCodeDir \ -link http://docs.oracle.com/javase/7/docs/api/ [http://docs.oracle.com/javase/7/docs/api/]`
- IntelliJ: Tools→Generate JavaDoc
- Alte IDE-uri de Java ex. Eclipse

Exemplu de clasa Java documentata:

```
import java.io.*;

/**
* <h1>Add Two Numbers!</h1>
* The AddNum program implements an application that
* simply adds two given integer numbers and Prints
* the output on the screen.
* <p>
* <b>Note:</b> Giving proper comments in your program makes it more
* user friendly and it is assumed as a high quality code.
*
* @author  Zara Ali
* @version 1.0
* @since   2014-03-31
*/
public class AddNum {
   /**
   * This method is used to add two integers. This is
   * a the simplest form of a class method, just to
   * show the usage of various javadoc Tags.
   * @param numA This is the first paramter to addNum method
   * @param numB  This is the second parameter to addNum method
   * @return int This returns sum of numA and numB.
   */
   public int addNum(int numA, int numB) {
      return numA + numB;
   }

   /**
   * This is the main method which makes use of addNum method.
   * @param args Unused.
   * @return Nothing.
   * @exception IOException On input error.
   * @see IOException
   */

   public static void main(String args[]) throws IOException {
      AddNum obj = new AddNum();
      int sum = obj.addNum(10, 20);

      System.out.println("Sum of 10 and 20 is :" + sum);
   }
}
```

Analiza codului

Pe langa verificarea functionalitatii codului prin testare, un alt mod de a cuantifica calitatea/performanta acestuia este prin a utiliza unelte de analiza. Aceste unelte pot face:

- Analiza statica
  - Fara a rula codul
  - Verifica coding style-ul folosit
  - Verifica data flow-ul programului prin reprezentarea programului ca un graf
- Analiza dinamica
  - Ruleaza codul
  - Urmareste utilizarea memoriei pentru a detecta memory leaks sau alte probleme
  - Masoara performanta prin profiling

Continuous Integration

In cadrul Continuous integration practicile de version control, compilarea si testarea automata sunt combinate astfel incat schimbarile care ajung in repository-ul de version control sunt automat rebuild-ate, testate si regenerate rapoartele aferente. Este de preferat sa se faca aceste lucruri complet sau partial pentru a reduce riscul ca schimbarile noi sa introduca probleme ca erori de compilare sau bug-uri.
Pentru ca sunt multi pasi necesari in a executa un plan de Continuous Integration, durata poate sa fie extrem de mare. Datorita acestui fapt, de fiecare data cand se introduc schimbari noi se va prefera rularea unui plan redus(de exemplu care include compilarea si analiza statica, dar nu include testare si analiza dinamica). Planul complet poate sa fie rulat ocazional, cum ar fi peste noapte, cand poate sa dureze mult mai mult. Pentru a putea beneficia de Continuous Integration, un proiect trebuie sa aiba mai multe caracteristici:

- Sistem de control al versiuni cu branch principal clar identificat sau set de branch-uri de development
- Build automat
- Dezvoltatori care introduc schimbari frecvent (poate de mai multe ori pe zi)
    - Commit-uri introduse pe branch-uri urmarite (ex. cu pull-request asociat) sunt verificate cu un plan minimal de Continuous Integration (compiling, static analysis)
- Testarea se face, ideal, pe o clona a mediului de productie
    - Diferit de mediul de dezvoltare
    - De obicei verificat rar
    - Poate sa foloseasca mai multe masini de rulare pentru a putea viza sisteme cu configuratii diferite sau pentru a paraleliza volumul de munca

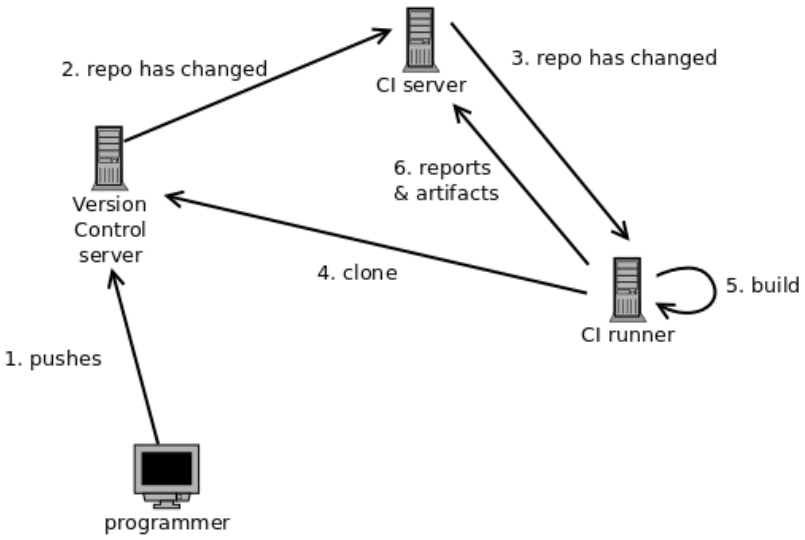
Avantaje:

- Problemele de integrare sunt identificate si rezolvate repede
- Schimbarile sunt testate cat mai repede posibil
- Se pune accentul pe check-in-uri frecvente, astfel se incurajeaza modularitatea

Dezavantaje:

- Efort initial ridicat pentru a face setup la sistem
- In functie de structura proiectului, e necesar un anumit nivel de rafinament pentru a pune fiecare pas de verificare intr-un build automat

Sistem de Continuous Integration



Un server de Continuous Integration este o masina accesibila din retea care:

- Are accesul la informatiile legate de proiectele care trebui rulate, locatia si informatiile de acces la repository-ul de version control, ce branch-uri sa urmareasca, cum sa faca build la proiect si ce rapoarte trebuie sa produca
- Monitorizeaza repository-ul de version control pentru a vedea noi commit-uri
- Cand un push la un branch urmarit apare, server-ul de CI notifica un runner

Un CI runner (sau nod) e un proces pe o masina care:

- Are uneltele necesare pentru a face build la proiect
- Este gestionat de serverul de CI

Cand un runner este notificat de serverul de CI, acesta:

- Cloneaza branch-ul pe care se doreste rularea build-ului din repository-ul de version control
- Ruleaza build-ul
- Publica rapoartele rezultate build-ului
    - Poate publica si “artefacte”: fisiere generate care contin informatii aditionale cum ar fi versiunea uneltelor folosite in cadrul build-ului

GitHub Actions

GitHub furnizeaza o solutie integrata de Continuous Integration printr-un serviciu numit “Actions”.
Acest serviciu poate fi accesat din pagina proiectului, selectand tab-ul Actions, de exemplu [https://github.com/sjzeil/CoWeM/actions].
Pentru a creea un Action nou, este necesara creearea unui fisier de workflow la calea `.github/workflows/` din repository. Acest lucru se poate face atat local cat si din GitHub. Fisierul de workflow are extensia `.yml` pentru ca foloseste sintaxa YAML [https://en.wikipedia.org/wiki/YAML] si contine indicatii legate de cand si ce este executat in cadrul unui action. Un exemplu de fisier de workflow care face build la un proiect Java cu Apache Ant gasiti aici [https://github.com/actions/starter-workflows/blob/main/ci/ant.yml].
Fisierele ce pot rezulta in urma rularii anumitor pasi pot fi publicate ca artefacte si in cadrul GitHub Actions. Pentru a face acest lucru, puteti folosi urmatoarea sintaxa, dupa pasul de generare al fisierului dorit:

```
- name: Upload a Build Artifact
    uses: actions/upload-artifact@v3.0.0
    with:
      # Artifact name
      name: # optional
      # Destination path
      path: # optional
```

Generarea unui artefact de build este o extensie a sistemului oferit de GitHub. Aici [https://github.com/marketplace?type=actions] gasiti marketplace-ul cu toate extensiile disponibile pentru GitHub Actions.

Exercitii

1. Descarcati si importati in IntelliJ proiectul din arhiva lab8.zip.
2. Documentati API-ul clasei CircularLinkedList, inclusiv clasa in sine. Documentarea consta in adaugarea unei scurte descriere a ce face clasa/functia si a parametrilor aferenti.
3. Creati functia cu antetul `public Node getNextNode(Node currentNode);` in clasa CircularLinkedList. Aceasta va intoarce urmatorul nod din currentNode. Documentati aceasta functie.
4. Generati documentatia folosind JavaDoc.
5. Faceti push la schimbari in GitHub. Aveti grija sa includeti doar fisierele necesare.
6. In GitHub, creati un Action care compileaza si ruleaza proiectul. Acesta va trebui sa foloseasca versiunea 17 de Java din distributia “temurin”.
7. Extindeti actiunea pentru a publica executabilul proiectului (fisierul .jar) ca artefact.
8. Extindeti actiunea pentru a genera documentatia proiectului folosind JavaDoc si a publica directorul documentatiei generate ca artefact. Verificati ca documentatia generata este aceeasi cu cea generata local.

Este recomandat sa folositi un fisier .gitignore care va ignora fisierele ce nu sunt necesare a fi incarcate in repo (cum ar fi fisierele executabile compilate). Acest fisier trebuie adaugat in folder-ul repository-ului (preferabil in root-ul acestuia) si adaugat in remote cu urmatorul commit. Un exemplu de continut pentru acest fisier gasiti aici [https://pastebin.com/hdiHVEMS].

Inclus in proiect este fisierul `build.xml`. Acesta este un fisier de build pentru Apache Ant (unealta folosita pentru a face toate procesele legate de compilarea si rularea aplicatiilor Java). Acest fisier contine target-uri pentru a face build si run la aplicatia proiectului, cat si generare a documentatiei folosind JavaDoc. Aceste target-uri vor trebui folosite in cadrul Action-ului/fisierului de workflow din GitHub. Puteti verifica local aceste target-uri din IntelliJ din Build→Build

with Ant… daca aveti fisierul .xml in radacina proiectului. Target-urile se adauga la finalul comenzii de build din exemplu.
Urmariti acest fisier pentru a sti path-ul unde se vor genera fisierele necesare pentru a crea artefactele de build.

Referințe

- https://www.cs.odu.edu/~zeil/cs350/latest/Public/docgen/index.html [https://www.cs.odu.edu/~zeil/cs350/latest/Public/docgen/index.html]
- https://www.cs.odu.edu/~zeil/cs350/latest/Public/continuousInteg/index.html [https://www.cs.odu.edu/~zeil/cs350/latest/Public/continuousInteg/index.html]
- https://www.cs.odu.edu/~zeil/cs350/latest/Public/analysis/index.html [https://www.cs.odu.edu/~zeil/cs350/latest/Public/analysis/index.html]
- https://github.com/eugenp/tutorials/blob/master/data-structures/src/main/java/com/baeldung/circularlinkedlist/ [https://github.com/eugenp/tutorials/blob/master/data-structures/src/main/java/com/baeldung/circularlinkedlist/]
- https://ant.apache.org/manual/using.html [https://ant.apache.org/manual/using.html]
- https://ucsb-cs56-pconrad.github.io/topics/javadoc_publishing_to_github_pages_from_public_repo/ [https://ucsb-cs56-pconrad.github.io/topics/javadoc_publishing_to_github_pages_from_public_repo/]
- https://docs.github.com/en/actions/automating-builds-and-tests/building-and-testing-java-with-ant [https://docs.github.com/en/actions/automating-builds-and-tests/building-and-testing-java-with-ant]
- https://www.tutorialspoint.com/java/java_documentation.htm [https://www.tutorialspoint.com/java/java_documentation.htm]

icalc/laboratoare/laborator-08.txt · Last modified: 2022/05/19 17:39 by liviu.mitruta